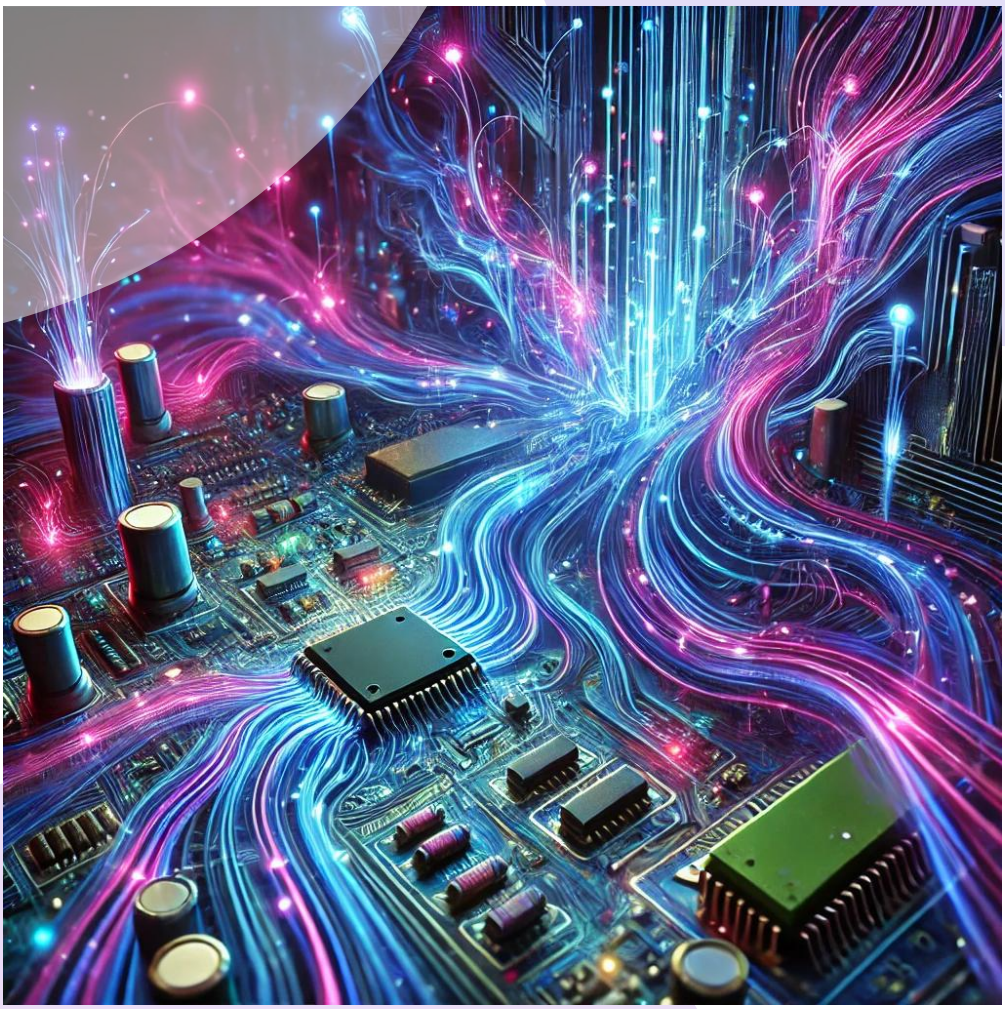


MINI PROJECT REPORT

Artificial Intelligence Course

TITLE:
Regression

Aylar Rahimzadeh
Fatemeh Sajed

هوش مصنوعی مقدماتی

فهرست

Titanic dataset

۱. آشنایی مجموعه داده ها
۲. پیش پردازش داده ها
۳. انتخاب ویژگی ، آموزش و ارزیابی

New DataSet

۱. دریافت دادگان
۲. پاکسازی و پیش پردازش داده ها برای سطر اول
۳. آموزش های مدل رگرسیون برای سطر اول
۴. پاکسازی و پیش پردازش داده ها برای سطر دوم
۵. آموزش های مدل رگرسیون برای سطر دوم
۶. پاکسازی و پیش پردازش داده ها برای سطر سوم
۷. آموزش های مدل رگرسیون برای سطر سوم
۸. پاکسازی و پیش پردازش داده ها برای قاب داده شامل سطر اول و دوم
۹. آموزش های مدل رگرسیون برای قاب داده شامل سطر اول و دوم

لینک های مربوطه

تایتانیک

https://colab.research.google.com/drive/1gxq1I0L7QfNzPFhi7r15o8fG6ImZuuex?usp=sharing

سطر اول داده جدید:

https://colab.research.google.com/drive/1icDi6jtAL6IwtylJdd-UHDgjX3wOOitO?usp=sharing

سطر دوم:

https://colab.research.google.com/drive/15oaiMjVNF2eWQEInn0qSwrxYu0q5dJRh?usp=sharing

سطر سوم:

https://colab.research.google.com/drive/1nI01Wc9nwLbBfG9bJN9cbLpcAdvE0E_o?usp=sharing

دیتافریم سطر۱و۲:

https://colab.research.google.com/drive/1LZd2VsUWt_Mdbajtsj4zHhVtOCcOjFvq?usp=sharing

۱. پیش پردازش داده ها

۱/۱. بررسی مجموعه داده ها

در این بخش به موارد زیر می پردازیم:

- **بدست اوردن ساختار کلی داده ها**
- **رسم نمودار correlation و بدست آوردن همبستگی ها**
- **رسم نمودار های پراکندگی و هگزبین**
- **رسم نمودار پراکندگی**

بدست آوردن ساختار کلی داده ها:

سطر اول کد این فایل مربوط به تمامی کتابخانه هایی است که در این تحلیل دیتاست مورد نیاز بوده است.

سطر بعدی در این فایل مربوط به خواندن دیتا می باشد که در متغیری ذخیره شده است تا بعدا بتوان از آن استفاده کرد و سپس برای نمایش آن با استفاده از تابع head() چند سطر ابتدایی را چاپ می کنیم.

سپس همان طور که در صورت سوال خواسته شده و مرسوم است باید اطلاعاتی کلی راجع به دیتایی که در حال تحلیل آن هستیم بدست بیاوریم بنابراین با استفاده از تابع describe() اطلاعاتی آماری از آن دریافت می کنیم. از مینیمم سطر ها و ستون ها بعدا در ادامه استفاده خواهد شد.

```
df.describe()
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

قدم بعدی آن است که درمورد نوع دیتای هر ستون تحقیق کنیم تا اگر دیتایی object است و قابلیت آن را دارد تا به یک اطلاعات آماری تبدیل شود را شناسایی کنیم بنابراین با استفاده از تابع detypes، آبجکت ها را شناسایی می کنیم.

```
df.dtypes
```

```
PassengerId      int64
Survived         int64
Pclass           int64
Name            object
Sex             object
Age            float64
SibSp            int64
Parch            int64
Ticket          object
Fare           float64
Cabin           object
Embarked        object
dtype: object
```

نکته مهم!!!!: در روند صورت سوال این گونه آمده است که ابتدا نمودار هایی رسم شود و سپس داده ها را پاک سازی کنیم اما از انجایی که گروه ها همفکری کرد که بهتر است داده های NAN ابتدا شناسایی شوند و داده کلیر شود و سپس به ادامه تحلیل بپردازیم همینجا داده های NAN را شناسایی کرده و درموردشان تصمیم میگیریم و سپس به رسم نمودار ها خواهیم پرداخت.

بنابراین باید داده های NAN را شناسایی کنیم برای این منظور از تابع isnull().sum() استفاده می کنیم که مجموع دیتا های not a number در هر ستون (ویژگی) به ما نشان می دهد.

```
: df.isnull().sum()
```

```
: PassengerId      0
  Survived         0
  Pclass           0
  Name             0
  Sex              0
  Age            177
  SibSp            0
  Parch            0
  Ticket           0
  Fare             0
  Cabin          687
  Embarked         2
  dtype: int64
```

خروجی نشان میدهد که ستون age , cabin دارای دیتاهای نامشخص هستند بنابراین برای آن ها باید چاره ای اندیشید تا تحلیل های آماری را خراب نکنند.؛ البته قبل از این index ها را تغییر می دهیم تا تعداد ستون های اضافه کمتر شوند.

```
df.rename(index=df['PassengerId'], inplace=True)
df.drop(['PassengerId'],axis=1,inplace=True)
df
```

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...
887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

891 rows × 11 columns

پس از این کار داده های نامشخص را به سه روش جایگزین می کنیم این روش کاملا دلخواه است و می تواند جور دیگری هم درنظر گرفته شود.

ستون سن: مقدارهای گمشده رو با میانگین سن‌ها پر می شوند.

ستون قیمت بلیط ها: با **میانه‌ی قیمت‌ها** پر می شوند (دلیل انتخاب: میانه در مقابل داده‌های پرت مقاومت است).

ستون بندرگاه: با **مد (mode)** یعنی پرتکرارترین مقدار پر می شوند.

- ستون شماره کابین: بیشتر مقادیر Cabin به شکل حروف + عدد هستند)
- این خط فقط **حرف اول** یعنی deck مربوطه رو نگه می‌داره. مثلاً C23 → C
- ولی اگذ مقدار "Unknown" باشه، همونطور باقی می مانند.

قدم بعدی آن است که ستون های متنی را به روشی تبدیل کنیم به داده هایی که از لحاظ آماری بتوان مورد بررسی قرار داد پس با استفاده از one-hot encoding آن ها را به ستون های عددی تبدیل می کنیم.

قدم بعدی نرمال سازی ویژگی های عددی است.

چرا این کار را می کنیم؟

چون زمانیکه دامنه داده های عددی مورد بررسی بسیار متفاوت باشد مقایسه نمودار ها و ویژگی های آنها دشوار و با خطا خواهد بود. بنابراین با استفاده از تابع get dummies() می توانیم گروهبندی ها را مشخص کنیم.

اینجا با استفاده از StandardScaler استاندارد سازی می کنیم و انحراف معیار ۱ است تا در مدل ها تاثیر برابر باشد.

	Survived	Pclass	Name	Age	SibSp	Parch	Ticket	Fare	Sex_male	Embarked_Q	Embarked_S	Cabin_B	Cabin_C	Cabin_D	Cabin_E	Cabin_F	Ca
1	0	3	Braund, Mr. Owen Harris	-0.592481	1	0	A/5 21171	-0.502445	True	False	True	False	False	False	False	False	
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	0.638789	1	0	PC 17599	0.786845	False	False	False	False	True	False	False	False	
3	1	3	Heikkinen, Miss. Laina	-0.284663	0	0	STON/O2. 3101282	-0.488854	False	False	True	False	False	False	False	False	
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0.407926	1	0	113803	0.420730	False	False	True	False	True	False	False	False	
5	0	3	Allen, Mr. William Henry	0.407926	0	0	373450	-0.486337	True	False	True	False	False	False	False	False	

	Age	SibSp	Parch	Ticket	Fare	Sex_male	Embarked_Q	Embarked_S	Cabin_B	Cabin_C	Cabin_D	Cabin_E	Cabin_F	Cabin_G	Cabin_T	Cabin_Unknown
	-0.592481	1	0	A/5 21171	-0.502445	True	False	True	False	False	False	False	False	False	False	True
	0.638789	1	0	PC 17599	0.786845	False	False	False	False	True	False	False	False	False	False	False
	-0.284663	0	0	STON/O2. 3101282	-0.488854	False	False	True	False	False	False	False	False	False	False	True
	0.407926	1	0	113803	0.420730	False	False	True	False	True	False	False	False	False	False	False
	0.407926	0	0	373450	-0.486337	True	False	True	False	False	False	False	False	False	False	True

در اینجا می توانیم ستون اسم ها را حذف کنیم چون اطلاعات خاصی از آن استخراج نمی شود و هیچ تاثیر آماری نه در آزمون و ارزیابی دارد و نه در بررسی ها تاثیری دارد.

```
df=df.drop(['Name'],axis=1,inplace=True)
df
```

رسم نمودار correlation و بدست آوردن همبستگی ها

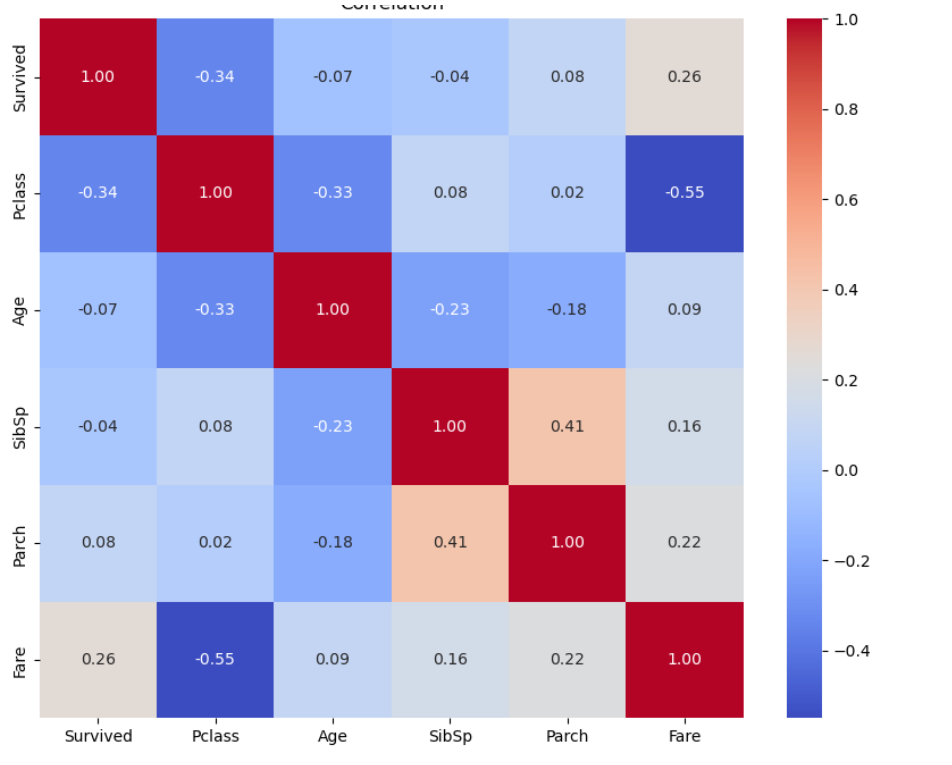
برای رسم نمودار همبستگی باید ابتدا داده های عددی را بدس بیاوریم برای اینکار از تابع select datatypes استفاده می کنیم درنتیجه یک دیتافریم داریم که فقط شامل ویژگی های عددی دیتای تایتانیک است. پس از آن با استفاده از تابع corr() ماتریس همبستگی را خساب می کنیم درواقع بین همه ستون ها ارتباط را نشان می دهد.

توضیحات:

مقدار هر سلول نشان می‌دهد که دو ویژگی عددی چقدر به هم وابسته هستند:

- **مقدار نزدیک به ۱: همبستگی مثبت قوی**
- **مقدار نزدیک به -۱: همبستگی منفی قوی**
- **مقدار نزدیک به ۰: بدون همبستگی**

سپس به سراغ رسم نمودار می رویم برای این کار از نمودار heatmap استفاده می کنیم و دلیل آن این ایست که می توان طیف ها را به خوبی با استفاده از رنگ ها نشان داد برای اینکار تنظیمات مربوطه مانند نمایش مقادیر داخل خانه های جدول و انتخاب پالت (تنالیته موردنظر برای نشان دادن رنگ ها) مشخص میکنیم.



تحلیل خروجی:

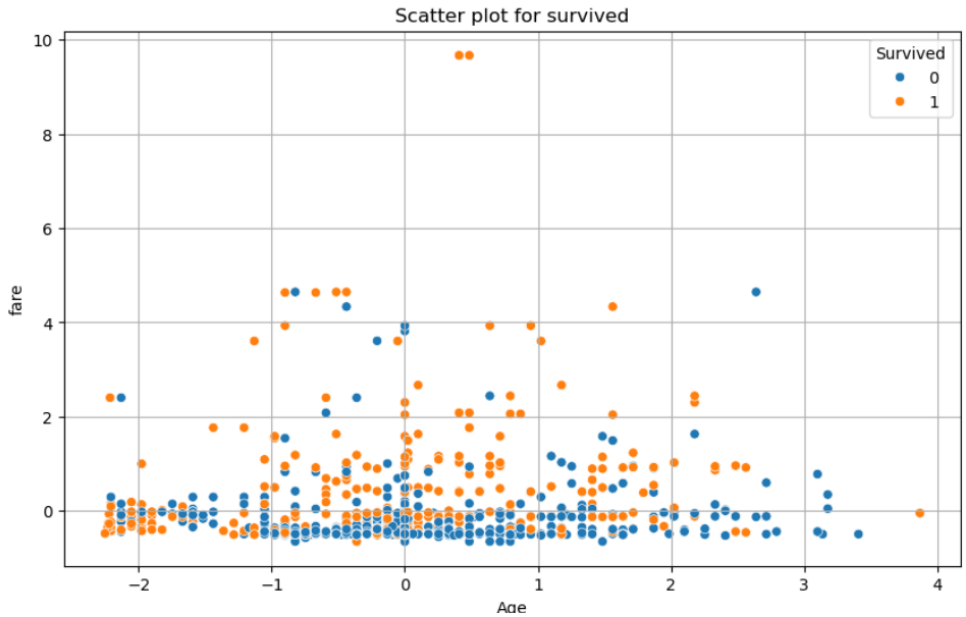
ویژگی ۱	ویژگی ۲	ضریب همبستگی	نوع همبستگی	تفسیر
Survived	Pclass	-0.34	منفی متوسط	احتمال نجات در کلاس‌های بالاتر بیشتر بوده است.
Survived	Age	-0.07	تقریباً صفر	سن تأثیر قابل توجهی بر نجات نداشته است.
Survived	SibSp	-0.04	تقریباً صفر	تعداد خواهر/برادر همراه تأثیر کمی بر نجات داشته است.
Survived	Parch	0.08	تقریباً صفر	تعداد والدین/فرزندان همراه تأثیر کمی بر نجات داشته است.
Survived	Fare	0.26	مثبت ضعیف	پرداخت کرایه بیشتر با احتمال بیشتر نجات همراه بوده است.
Pclass	Age	-0.33	منفی متوسط	مسافران کلاس پایین‌تر معمولاً جوان‌تر بوده‌اند.
Pclass	SibSp	0.08	تقریباً صفر	رابطه خاصی بین کلاس سفر و خواهر/برادر دیده نمی‌شود.
Pclass	Parch	0.02	تقریباً صفر	رابطه خاصی بین کلاس سفر و والدین/فرزندان دیده نمی‌شود.
Pclass	Fare	-0.55	منفی نسبتاً قوی	کلاس پایین‌تر معمولاً با کرایه کمتر همراه بوده است.
Age	SibSp	-0.23	منفی ضعیف	افراد مسن‌تر معمولاً تنها سفر کرده‌اند.
Age	Parch	-0.18	منفی ضعیف	افراد مسن‌تر کمتر با والدین/فرزندان همراه بوده‌اند.
Age	Fare	0.09	تقریباً صفر	سن و کرایه پرداختی تقریباً ارتباطی ندارند.
SibSp	Parch	0.41	مثبت نسبتاً قوی	کسانی که خواهر/برادر داشته‌اند، احتمال همراهی با والدین/فرزندان بیشتر بوده است.

تعداد خواهر/برادر همراه، تا حد کمی با کرایه بیشتر همراه بوده است.	مثبت ضعیف	0.16	Fare	*SibSp*
تعداد والدین/فرزندان همراه، کمی با کرایه بیشتر همراه بوده است.	مثبت ضعیف	0.22	Fare	*Parch*

نمودار پراکندگی

در اینجا ابتدا صفحه نمایش نمودار را آماده می کنیم و سپس با استفاده از scatterplot به رسم نمودار نقطه ای می پردازیم در این تنظیمات ابتدا محور x,y را مشخص می کنیم که طبق خواسته سوال باید هزینه و سن باشد و از hue استفاده می کنیم تا بر اساس اینکه نجات پیدا کرده اند یا نه رنگ داده ها را نمایش دهد. پس از تنظیماتی مثل نام محور ها و نام نمودار باید راهنمای نمودار را هم فعال کنیم تا تفکیک رنگ بر اساس نجات پیدا کردن مشخص شود.

[84]:
```
# نمودار پراکندگی (scatter)
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x="Age", y="Fare", hue="Survived")
plt.title("Scatter plot for survived")
plt.xlabel("Age")
plt.ylabel("fare")
plt.legend(title="Survived")
plt.grid(True)
plt.show()
```



تحلیل خروجی :

فراد نجات‌یافته (نقاط نارنجی) بیشتر در نواحی با کرایه‌های بالاتر قرار دارند، مخصوصاً در بازه سنی میانه.

افراد نجات‌نیافته (نقاط آبی) بیشتر در نواحی با کرایه پایین متمرکز هستند.

سن (Age) تأثیر واضحی در نجات ندارد، چون در هر رنج سنی هم زنده‌مانده و هم فوت‌شده وجود دارد.

تراکم داده‌ها در سن نزدیک به میانگین (۰ پس از استانداردسازی) و کرایه نزدیک به ۰ بالاست (بیشتر افراد در طبقه متوسط یا پایین بوده‌اند)

رسم نمودار هگزبین برای سن و هزینه

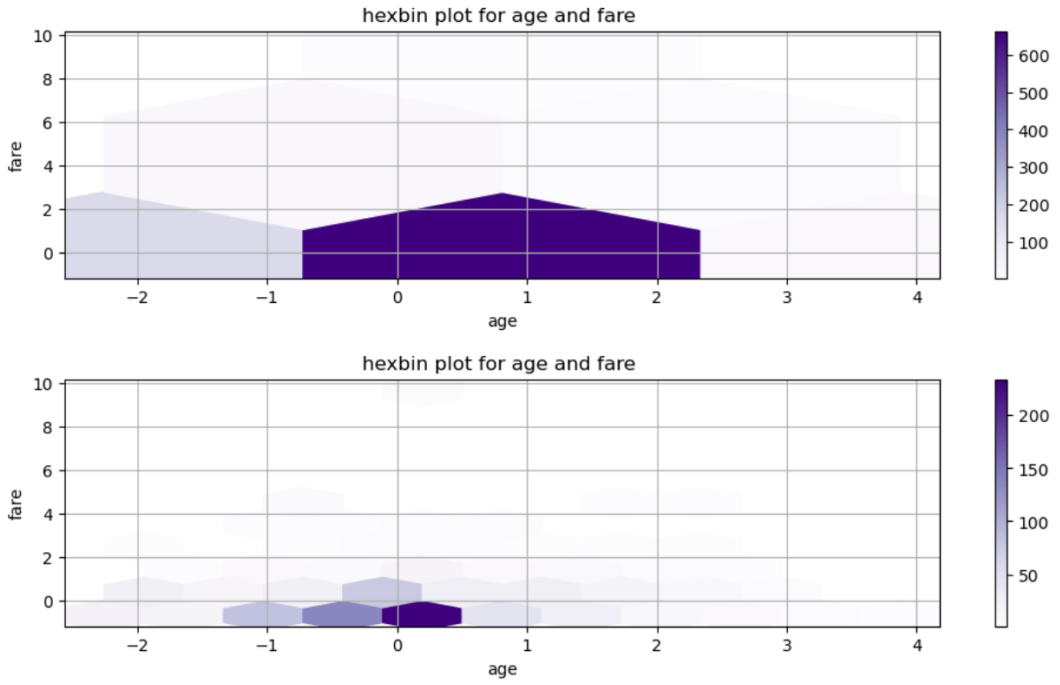
```
# 4. نمودار هگزبین
plt.figure(figsize=(12, 6))
plt.subplot(2,1,1)
plt.hexbin(df["Age"], df["Fare"], gridsize=2, cmap='Purples', mincnt=1)
plt.colorbar()
plt.xlabel("age")
plt.ylabel("fare")
plt.title("hexbin plot for age and fare ")
plt.grid()

plt.figure(figsize=(12, 6))
plt.subplot(2,1,2)
plt.hexbin(df["Age"], df["Fare"], gridsize=10, cmap='Purples', mincnt=1)
plt.colorbar()
plt.xlabel("age")
plt.ylabel("fare")
plt.title("hexbin plot for age and fare ")
plt.grid(True)

plt.show()
```

از انجایی که این اعداد یک بار اسکیل شده اند خیلی داده دقیقی نمیتوان بدست اورد اما می توان به طور شهودی آنرا مورد بررسی قرار داد در کد این رسم نمودار، با دو مقیاس نمودار هگزبین را رسم کردیم چون داده ها اسکیل شده و حول ۰ و یک و و.. هستند.

مقیاس بندی را با استفاده از grid size مشخص کردیم و محور های x,y را مشخص کردیم و بااستفاده از subplot دو نمودار را در یک تصویر نشان می دهیم.



رسم نمودار پراکندگی نقطه ای بر اساس جنسیت

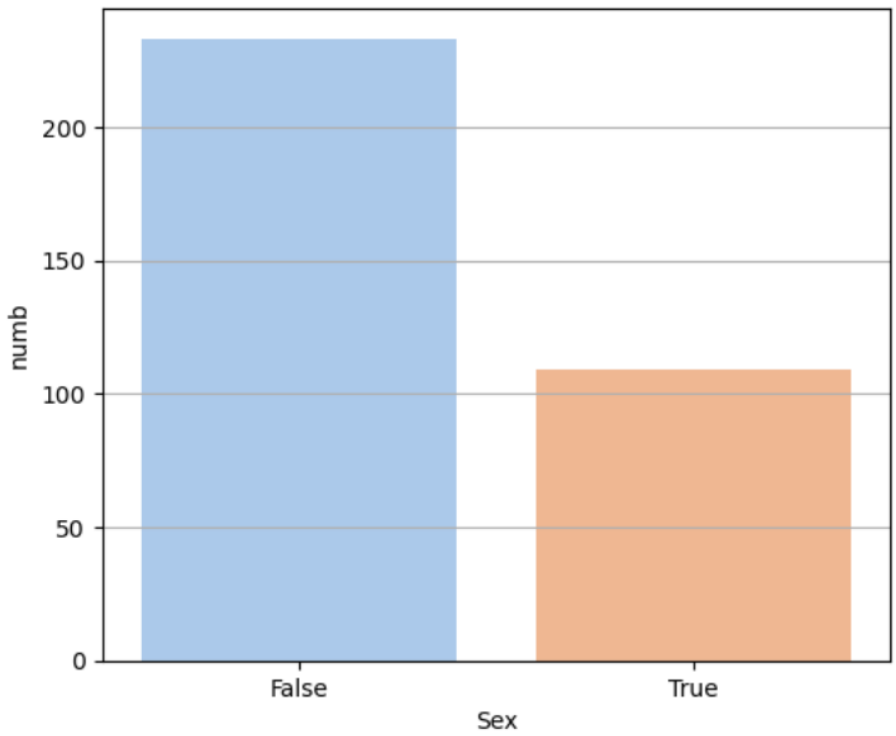
```
[15]: #  6. نمودار countplot برای جنسیت نجات‌یافتگان
plt.figure(figsize=(6, 5))
sns.countplot(data=survived_df, x="Sex_male", palette="pastel")
plt.xlabel("Sex")
plt.ylabel("numb")
plt.grid(True, axis='y')
plt.show()
```

در این بخش نمودار نقطه ای را بر اساس

رسم نمودار پراکندگی هزینه و سن با استفاده از تابع countplot تعداد داده ها را میشمارد و رسم می کند.

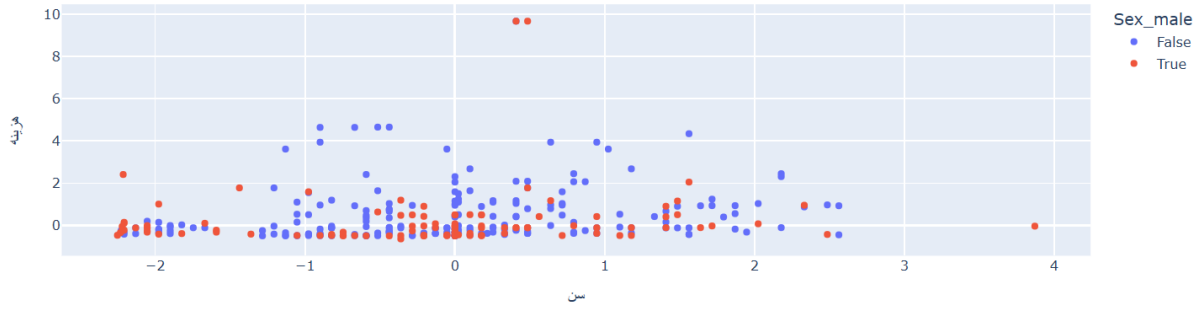
خروجی:

```
#  6. نمودار countplot برای جنسیت نجات‌یافتگان
plt.figure(figsize=(6, 5))
sns.countplot(data=survived_df, x="Sex_male", palette="pastel")
plt.xlabel("Sex")
plt.ylabel("numb")
plt.grid(True, axis='y')
plt.show()
```



```
survived_df=df[df['Survived']==1]

fig=px.scatter(survived_df,x='Age',y='Fare',labels={'Age': "سن",'Fare': 'هزینه'}, hover_data=['Sex_male', 'Pclass'],color='Sex_male')
fig.show()
```



تمرکز نقاط در کرایه پایین:

- بیشتر نجات‌یافتگان در بازه‌ی هزینه نزدیک به ۰ تا ۲ (بعد از استانداردسازی) قرار دارند.
- این نشان می‌دهد نجات صرفاً مربوط به افراد ثروتمند نبوده، هرچند در بخش هزینه بلیط بالا نیز نجات‌یافته‌هایی دیده می‌شوند.

توزیع سنی متنوع:

- نجات‌یافتگان در تمام رده‌های سنی حضور دارند، اما تراکم بیشتری در اطراف میانگین سن (حدود ۰) وجود دارد.

مقایسه بر اساس جنسیت:

- تعداد بیشتری از نجات‌یافتگان را **زنان (نقاط آبی)** تشکیل می‌دهند.
- این موضوع با واقعیت تاریخی سیاست "اول زنان و کودکان" در تایتانیک تطابق دارد.

هزینه های بالا بیشتر توسط زنان پرداخت شده:

- در نواحی با کرایه بالا، اغلب نقاط به رنگ آبی هستند که نشان‌دهنده‌ی سهم بیشتر زنان در بین نجات‌یافتگان با بلیط‌های گران‌تر است.

۱/۲. تحلیل آماری مجموعه داده ها

تعداد اعضای خانواده

```
[ ]: #  محاسبه تعداد اعضای خانواده
df['FamilySize'] = df['SibSp'] + df['Parch']

# گروه‌بندی بر اساس اندازه خانواده و نرخ بقا
family_size_survival = df.groupby('FamilySize')['Survived'].mean()
#print("Impact of Family Size on Survival Rate:")
family_size_survival
```

```
[ ]: FamilySize
0     0.303538
1     0.552795
2     0.578431
3     0.724138
4     0.200000
5     0.136364
6     0.333333
7     0.000000
10    0.000000
Name: Survived, dtype: float64
```

توضیح:

SibSp: تعداد خواهر/برادر یا همسر همراه.

Parch: تعداد پدر/مادر یا فرزند همراه.

جمع این دو نشان‌دهنده تعداد اعضای خانواده همراه در سفر است.

گروه‌بندی داده‌ها بر اساس اندازه خانواده (FamilySize) و سپس محاسبه میانگین مقدار Survivedبرای هر گروه.

چون Survived فقط ۰ یا ۱ است، میانگین آن برابر با نرخ نجات آن گروه خواهد بود.

تحلیل خروجی:

خانواده‌های کوچک (۱ تا ۳ نفر) بیشترین شانس نجات را داشته‌اند.

افراد تنها یا خانواده‌های خیلی بزرگ نرخ نجات پایین‌تری داشته‌اند.

می‌توان نتیجه گرفت که در شرایط بحرانی (مثل تایتانیک)، وجود حداقل یک همراه مفید است، اما همراهی بیش از حد باعث کاهش شانس نجات می‌شود، احتمالاً به دلیل دشواری نجات گروه‌های بزرگ‌تر.

بررسی تاثیر سفر تنهایی:

```
# تأثیر سفر تنها یی
df['IsAlone'] = df['FamilySize'].apply(lambda x: 1 if x == 0 else 0)
alone_survival_rate = df.groupby('IsAlone')['Survived'].mean()
alone_survival_rate
```

```
IsAlone
0    0.505650
1    0.303538
Name: Survived, dtype: float64
```

توضیح:

برای این قسمت نیز زمانی که شماره خانواده ۱ باشد انرا تنها می نامیم پس باید میزان نجات یافتگانی که شماره خانواده شان ۱ است میانگین گیری شود.

خروجی نشان می دهد که بیشتر افراد تنها جان باخته اند.

```
#اضافه کردن شانس بقا  بر اساس اندازه خانواده
df['FamilySizeSurvivalRate'] = df['FamilySize'].map(family_size_survival)

#: اضافه کردن شانس بقا  بر اساس سفر تنها یی
df['AloneSurvivalRate'] = df['IsAlone'].map(alone_survival_rate)

# اضافه کردن شانس بقا  بر اساس گروه سنی
df['AgeGroupSurvivalRate'] = df['AgeGroup'].map(age_group_survival)
print(df[['FamilySize', 'FamilySizeSurvivalRate', 'IsAlone', 'AloneSurvivalRate', 'AgeGroup', 'AgeGroupSurvivalRate']].head())
```

```
   FamilySize  FamilySizeSurvivalRate  IsAlone  AloneSurvivalRate  \
1           1                0.552795        0           0.505650
2           1                0.552795        0           0.505650
3           0                0.303538        1           0.303538
4           1                0.552795        0           0.505650
5           0                0.303538        1           0.303538

      AgeGroup AgeGroupSurvivalRate
1        teen            0.344168
2  Yuong Adult            0.404255
3        teen            0.344168
4  Yuong Adult            0.404255
5  Yuong Adult            0.404255
```

در این قسمت از کد به جمع بندی و نمایش به صورت جدول می پردازیم برای اینکار از تابع map کردن استفاده می کنیم در مپ کردن دیتاهایی که داریم را متناظرا به مکانی که مدنظر داریم منتقل می کنیم. در اینجا هم ستون های جدید درست می کنیم و مقادیر بدست امده را متناظرا وارد دیتاست می کنیم.

خروجی:

نتیجه گیری: افراد تنها شانس بقای بالایی ندارند.

دسته young adult نرخ بالای بقای بیشتری نسبت به نوجوانان دارند. ولی به صورت کلی سنین مختلف نرخ بقای متفاوتی دارند.

۲. پیش پردازش داده ها

همانطور که قبل تر توضیح داده شد اسکیل کردن و نرمال سازی و جایگزینی داده های نامشخص در قسمت قبل و در ابتدا صورت گرفته است اینجا صرفا به سوالات توضیحی پاسخ داده خواهد شد.

تفاوت ویژگی‌های عددی و دسته‌ای

نوع داده در پایتون	مثال‌ها	تعریف	*نوع ویژگی*
int, float	Fare ،Age	ویژگی‌هایی با مقادیر عددی که امکان انجام عملیات ریاضی روی آن‌ها وجود دارد.	*عددی (Numerical)*
object، string	Embarked ،Sex، Cabin	ویژگی‌هایی که نمایانگر طبقه‌ها یا گروه‌های غیرعددی هستند.	*دسته‌ای (Categorical)*

نوع ویژگی‌ها در داده‌های مجموعه تایتانیک

توضیحات	نوع داده در پایتون	نوع ویژگی	*نام ویژگی*
نیاز به پر کردن داده‌های گمشده و استانداردسازی دارد	float64	عددی	*Age*
نیاز به پر کردن داده‌های گمشده و استانداردسازی دارد	float64	عددی	*Fare*
نیاز به رمزگذاری (One-Hot Encoding) دارد	object	دسته‌ای (متنی)	*Sex*
پر کردن مقادیر گمشده با مد و سپس رمزگذاری	object	دسته‌ای (متنی)	*Embarked*
استخراج حرف اول کابین و رمزگذاری	object	دسته‌ای (متنی)	*Cabin*
بسته به تحلیل ممکن است به صورت دسته‌ای در نظر گرفته شود	int64	عددی با ماهیت طبقه‌ای	*Pclass*

پیش‌پردازش مناسب برای ویژگی‌های دسته‌ای

توضیح	*نوع پیش‌پردازش*
معمولاً با مد (پرتکرارترین مقدار) انجام می‌شود.	*پر کردن مقادیر گمشده*
استفاده از One-Hot Encoding یا Label Encoding برای تبدیل مقادیر متنی به عددی.	*رمزگذاری دسته‌ای (Encoding)*

مقایسه نرمال‌سازی و استانداردسازی

روش	تعریف	دامنه خروجی	کاربرد مناسب
نرمال‌سازی (Normalization)	تبدیل مقادیر به بازه[0, 1]	[0, 1]	برای داده‌هایی با مقیاس متفاوت یا توزیع غیرنرمال
استانداردسازی (Standardization)	تنظیم میانگین به صفر و انحراف معیار به یک	بدون دامنه مشخص	برای مدل‌هایی حساس به مقیاس مانندSVM ، KNN

مدل رگرسیون خطی به مقیاس ویژگی ها نیز حساس است بنابراین استانداردسازی نیاز است. کاری که قبلا انجام شده است.

۳. آزمون و ارزیابی و انتخاب ویژگی

انتخاب ویژگی

دراینجا باید ویژگی هایی را انتخاب کنیم. بنابراین، ابتدا ویژگی‌های جدیدی به مجموعه داده افزوده شده‌اند. هدف از این مرحله، استخراج اطلاعات ترکیبی از ویژگی‌های موجود برای تقویت عملکرد مدل یادگیری ماشین است. ویژگی اول، نسبت سن به کرایه (Age_Fare_Ratio)است که ممکن است نشان‌دهنده نوع سفر یا سطح اقتصادی باشد. ویژگی دوم، اندازه خانواده (Family_Size)است که با جمع تعداد خواهر/برادر و والدین/فرزندان به‌دست می‌آید. ویژگی سوم، متغیر Is_Aloneاست که مشخص می‌کند آیا مسافر به‌تنهایی سفر کرده یا با همراه بوده است.

در مرحله بعد، الگوریتم رگرسیون لاسو (Lasso Regression) برای انتخاب ویژگی‌ها استفاده شده است. ابتدا داده‌ها آماده‌سازی شده‌اند و ستون‌های غیرضروری مانند متغیر هدف(Survived) ، شماره بلیت، و دسته‌بندی سنی حذف شده‌اند. پس از تقسیم داده‌ها به مجموعه‌های آموزش و آزمون، مدل لاسو با استفاده از اعتبارسنجی متقابل آموزش دیده است. لاسو با استفاده از جریمه L1 ، برخی ویژگی‌ها را حذف کرده و تنها ویژگی‌های مؤثر را حفظ می‌کند. ویژگی‌هایی که ضریب آن‌ها صفر نشده است، به‌عنوان ویژگی‌های مهم در نظر گرفته می‌شوند.

در مرحله سوم، روش RFE (حذف بازگشتی ویژگی‌ها) با استفاده از مدل پایه رگرسیون لجستیک اجرا شده است. این روش به‌صورت مرحله‌ای، ضعیف‌ترین ویژگی‌ها را حذف می‌کند تا تنها تعداد مشخصی از بهترین ویژگی‌ها باقی بمانند. در اینجا پنج ویژگی انتخاب شده‌اند که بیشترین تأثیر را بر پیش‌بینی متغیر هدف داشته‌اند.

در پایان، اشتراک بین ویژگی‌های انتخاب‌شده توسط دو روش لاسو و RFE گرفته شده و به‌عنوان ویژگی‌های نهایی مؤثر معرفی شده‌اند. این تقاطع کمک می‌کند تا تنها متغیرهایی که از نظر هر دو روش دارای اهمیت هستند، برای مدل نهایی مورد استفاده قرار گیرند. این روش ترکیبی به بهبود دقت مدل و کاهش پیچیدگی آن کمک می‌کند.

کد مربوطه:

```
#  ایجاد ویژگی‌های جدید
df["Age_Fare_Ratio"] = df["Age"] / df["Fare"]
df["Family_Size"] = df["SibSp"] + df["Parch"]
df["Is_Alone"] = (df["Family_Size"] == 0).astype(int)

X = df.drop(columns=["Survived", "Ticket",'AgeGroup','AgeGroupSurvivalRate'])
y = df["Survived"]

# تقسیم داده‌ها  به داده‌های آموزش و تست
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=43)

# مدل رگرسیون لاسو
lasso = LassoCV()
lasso.fit(X_train, y_train)

# نمایش ویژگی‌های انتخاب شده  توسط لاسو
lasso_selected_features = X.columns[lasso.coef_ != 0]
print("Features selected by Lasso:", lasso_selected_features)

#  حذف ویژگی‌ها  به صورت بازگشتی (RFE)
log_reg = LogisticRegression(max_iter=1000)

rfe = RFE(log_reg, n_features_to_select=5)  # انتخاب 5 ویژگی
rfe.fit(X_train, y_train)
rfe_selected_features = X.columns[rfe.support_]
print("Features selected by RFE:", rfe_selected_features)

#  4. مقایسه ویژگی‌های انتخاب شده

final_selected_features = list(set(lasso_selected_features) & set(rfe_selected_features))
print("Final selected features:", final_selected_features)
```

```
Features selected by Lasso: Index(['Pclass', 'Age', 'SibSp', 'Parch', 'Fare', 'Sex_male', 'Embarked_Q',
       'Embarked_S', 'Cabin_B', 'Cabin_C', 'Cabin_D', 'Cabin_E', 'Cabin_F',
       'Cabin_G', 'Cabin_Unknown', 'IsAlone', 'FamilySizeSurvivalRate',
       'Age_Fare_Ratio', 'Is_Alone'],
      dtype='object')
Features selected by RFE: Index(['Pclass', 'Sex_male', 'Cabin_G', 'FamilySizeSurvivalRate', 'Is_Alone'], dtype='object')
Final selected features: ['Is_Alone', 'Pclass', 'Sex_male', 'Cabin_G', 'FamilySizeSurvivalRate']
```

توضیح خروجی:

۱ .ویژگی‌های انتخاب‌شده توسط لاسو(Lasso)

مدل لاسو ویژگی‌هایی را انتخاب کرده که ضرایب آن‌ها صفر نبوده‌اند. این ویژگی‌ها شامل موارد زیر هستند:

- ویژگی‌های عددی: Pclass، Age، Parch، Fare
- ویژگی‌های باینری یا رمزگذاری‌شده: Sex_male، Embarked_S
- ویژگی‌های مربوط به کابین: Cabin_B، Cabin_C، Cabin_D، Cabin_E، Cabin_F، Cabin_G، Cabin_Unknown
- ویژگی‌های مهندسی‌شده: FamilySize، IsAlone، FamilySizeSurvivalRate، Age_Fare_Ratio، Is_Alone، Family_Size

این انتخاب نشان می‌دهد که مدل لاسو بسیاری از متغیرهای خام و مهندسی‌شده را مؤثر در پیش‌بینی در نظر گرفته است.

۲ .ویژگی‌های انتخاب‌شده توسط RFE

روش RFE تنها پنج ویژگی برتر را انتخاب کرده است:

- Pclass
- Sex_male
- Cabin_E
- Cabin_F
- FamilySizeSurvivalRate

این ویژگی‌ها با استفاده از مدل رگرسیون لجستیک و ارزیابی تکراری انتخاب شده‌اند. ویژگی‌های انتخاب‌شده شامل اطلاعات مکانی (کابین)، اجتماعی (جنسیت و طبقه سفر) و آماری (نرخ بقا براساس اندازه خانواده) هستند.

۳ .ویژگی‌های نهایی (مشترک بین دو روش)

ویژگی‌هایی که هم توسط لاسو و هم توسط RFE انتخاب شده‌اند، به‌عنوان ویژگی‌های نهایی برای مدل‌سازی در نظر گرفته شده‌اند:

- Pclass
- Sex_male
- Cabin_E
- Cabin_F
- FamilySizeSurvivalRate

این ویژگی‌ها از نظر دو الگوریتم مستقل، بیشترین قدرت پیش‌بینی برای متغیر هدف (Survived) را دارند و می‌توانند به عنوان ورودی نهایی مدل انتخاب شوند.

رگرسیون Logistic و تحلیل های آن

کد:

```
# ایجاد مدل رگرسیون لجستیک
log_reg = LogisticRegression(max_iter=1000)

# مدل با ویژگی‌های انتخابی توسط لاسو
X_lasso = X[lasso_selected_features]  # ویژگی‌های انتخاب شده توسط لاسو
lasso_score = cross_val_score(log_reg, X_lasso, y, cv=5, scoring='accuracy').mean()
print(f"Mean accuracy with Lasso selected features: {lasso_score:.4f}")

# مدل با ویژگی‌های انتخابی توسط RFE
X_rfe = X[rfe_selected_features]  # ویژگی‌های انتخاب شده توسط RFE
rfe_score = cross_val_score(log_reg, X_rfe, y, cv=5, scoring='accuracy').mean()
print(f"Mean accuracy with RFE selected features: {rfe_score:.4f}")

# مدل با ویژگی‌های انتخابی توسط هر دو روش
X_final = X[final_selected_features]  # ویژگی‌های انتخاب شده توسط هر دو روش
final_score = cross_val_score(log_reg, X_final, y, cv=5, scoring='accuracy').mean()
print(f"Mean accuracy with final selected features (intersection of Lasso and RFE): {final_score:.4f}")
```

توضیح:

در این بخش، سه مدل **رگرسیون لجستیک** با استفاده از سه مجموعه مختلف از ویژگی‌ها آموزش داده شده‌اند تا عملکرد آن‌ها از نظر دقت (accuracy) مقایسه شود. ارزیابی مدل‌ها با استفاده از **اعتبارسنجی متقابل پنج‌تایی** انجام شده است.

۱. **ساخت مدل پایه**

- یک نمونه از رگرسیون لجستیک ساخته شده با تعداد تکرارهای بیشتر از حالت پیش‌فرض (برای اطمینان از همگرایی مدل).

۲. **مدل‌سازی با ویژگی‌های انتخاب‌شده توسط لاسو**

- ابتدا زیرمجموعه‌ای از داده‌ها شامل فقط ویژگی‌هایی که توسط لاسو انتخاب شده‌اند، ایجاد شده است.
- سپس دقت مدل با استفاده از اعتبارسنجی متقابل محاسبه شده و میانگین آن ثبت شده است.

۳. **مدل‌سازی با ویژگی‌های انتخاب‌شده توسط RFE**

- مشابه مرحله قبل، اما با استفاده از ویژگی‌هایی که توسط روش حذف بازگشتی (RFE) انتخاب شده‌اند.
- دقت میانگین این مدل نیز با اعتبارسنجی متقابل اندازه‌گیری شده است.

۴. **مدل‌سازی با اشتراک دو روش (ویژگی‌های نهایی)**

- در این بخش، فقط ویژگی‌هایی که هم توسط لاسو و هم توسط RFE انتخاب شده‌اند، استفاده شده است.
- این ویژگی‌ها به‌عنوان ترکیب نهایی انتخاب ویژگی در نظر گرفته شده‌اند و مدل نهایی با استفاده از آن‌ها ساخته شده است.

خروجی:

```
Mean accuracy with Lasso selected features: 0.8114
Mean accuracy with RFE selected features: 0.7958
Mean accuracy with final selected features (intersection of Lasso and RFE): 0.7958
```

بالاترین دقت مربوط به ویژگی‌های انتخاب‌شده توسط لاسو است (حدود ۸۰. درصد). این نشان می‌دهد که لاسو توانسته ویژگی‌های مؤثرتری برای طبقه‌بندی انتخاب کند.

ویژگی‌های انتخاب‌شده توسط RFE و اشتراک بین دو روش، دقت یکسانی دارند)حدود ۷۹۵ درصد.(

با وجود اینکه روش ترکیبی در برخی مواقع برای کاهش پیچیدگی مدل مفید است، در این مورد **عملکرد مدل نهایی به خوبی مدل بر اساس ویژگی‌های لاسو نیست.**

مدل رگرسیون باینری:

کد

```
# مدل رگرسیون باینری
log_reg = LogisticRegression(solver='liblinear')

# انتخاب ویژگی‌ها
X_lasso = X[lasso_selected_features]
X_rfe = X[rfe_selected_features]
X_final = X[final_selected_features]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=43)
```

```
# آموزش مدل با ویژگی‌های انتخابی توسط لاسو
log_reg.fit(X_train, y_train)
y_pred_lasso = log_reg.predict(X_test)
accuracy_lasso = accuracy_score(y_test, y_pred_lasso)
```

```
# آموزش مدل با ویژگی‌های انتخابی توسط RFE
log_reg.fit(X_train, y_train)
y_pred_rfe = log_reg.predict(X_test)
accuracy_rfe = accuracy_score(y_test, y_pred_rfe)
```

```
# آموزش مدل با ویژگی‌های انتخابی توسط هر دو روش
log_reg.fit(X_train, y_train)
y_pred_final = log_reg.predict(X_test)
accuracy_final = accuracy_score(y_test, y_pred_final)
```

```
# محاسبه AUC و ROC برای هر مدل
fpr_lasso, tpr_lasso, _ = roc_curve(y_test, log_reg.predict_proba(X_test)[:,1])
roc_auc_lasso = auc(fpr_lasso, tpr_lasso)

fpr_rfe, tpr_rfe, _ = roc_curve(y_test, log_reg.predict_proba(X_test)[:,1])
roc_auc_rfe = auc(fpr_rfe, tpr_rfe)

fpr_final, tpr_final, _ = roc_curve(y_test, log_reg.predict_proba(X_test)[:,1])
roc_auc_final = auc(fpr_final, tpr_final)

# رسم نمودار ROC
plt.figure(figsize=(10, 6))
plt.plot(fpr_lasso, tpr_lasso, color='blue', lw=2, label=f'Lasso (AUC = {roc_auc_lasso:.2f})')
plt.plot(fpr_rfe, tpr_rfe, color='green', lw=2, label=f'RFE (AUC = {roc_auc_rfe:.2f})')
plt.plot(fpr_final, tpr_final, color='red', lw=2, label=f'Final (AUC = {roc_auc_final:.2f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.show()
```

توضیحات:

در اینجا. مدل‌ها از سه مجموعه ویژگی متفاوت استفاده می‌کنند و هدف این است که با مقایسه دقت (accuracy) و عملکرد مدل‌ها با استفاده از منحنی ROC و مساحت زیر آن(AUC) ، بهترین مدل انتخاب شود. این فرآیند شامل مراحل مختلف آموزش، ارزیابی و مقایسه مدل‌ها است.

ابتدا، یک مدل رگرسیون لجستیک با حل‌گر liblinearبرای مسائل باینری تعریف می‌شود. این مدل بر اساس سه مجموعه ویژگی متفاوت آموزش داده خواهد شد: اولین مجموعه ویژگی‌ها از روش لاسو (Lasso) انتخاب شده‌اند، دومی از طریق روش حذف بازگشتی ویژگی‌ها (RFE) انتخاب شده‌اند، و سومی ترکیبی از ویژگی‌هایی است که هم توسط لاسو و هم توسط RFE انتخاب شده‌اند. این ویژگی‌ها نمایانگر سه رویکرد مختلف در انتخاب ویژگی‌ها هستند

داده‌ها سپس به دو بخش تقسیم می‌شوند: داده‌های آموزشی و داده‌های آزمون. داده‌های آموزشی برای آموزش مدل‌ها استفاده می‌شوند، در حالی که داده‌های آزمون برای ارزیابی عملکرد مدل‌ها پس از آموزش به کار می‌روند. این تقسیم‌بندی به صورت تصادفی انجام می‌شود و با استفاده از مقدار ثابت random_state=43اطمینان حاصل می‌شود که این تقسیم‌بندی در هر بار اجرا یکسان باشد.

پس از تقسیم داده‌ها، مدل رگرسیون لجستیک برای هر سه مجموعه ویژگی آموزش داده می‌شود. برای هر مدل، پیش‌بینی‌هایی بر اساس داده‌های آزمون انجام شده و دقت (accuracy) مدل محاسبه می‌شود. دقت به‌عنوان یکی از معیارهای مهم برای ارزیابی مدل‌های طبقه‌بندی استفاده می‌شود. این مقادیر دقت برای هر مجموعه ویژگی چاپ می‌شوند تا مقایسه‌ای عددی بین آن‌ها انجام گیرد.

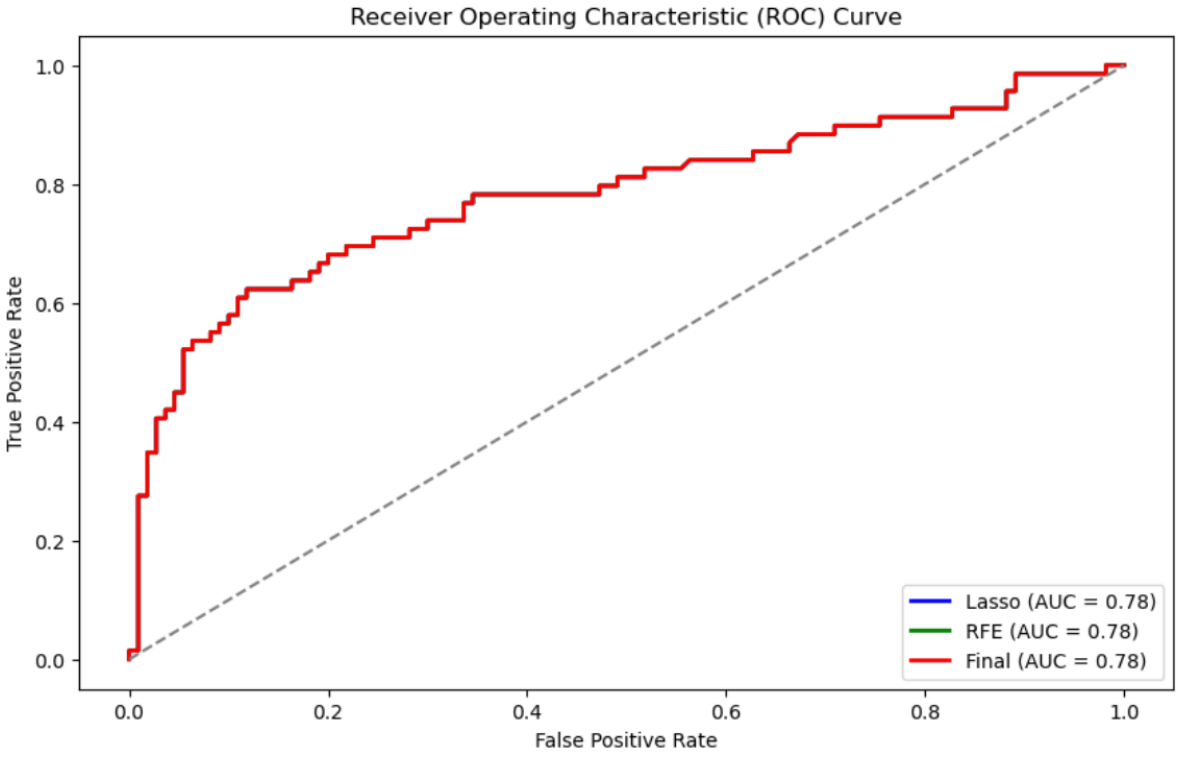
برای ارزیابی دقیق‌تر عملکرد مدل‌ها، منحنی ROC برای هر مدل ترسیم می‌شود. منحنی ROC رابطه بین نرخ مثبت کاذب (FPR)و نرخ مثبت واقعی (TPR) را نشان می‌دهد و به ما کمک می‌کند تا ببینیم مدل در تمایز بین کلاس‌های مثبت و منفی چه عملکردی دارد. به علاوه، مساحت زیر این منحنی (AUC) برای هر مدل محاسبه شده و به عنوان معیاری از کیفیت مدل در تمایز بین کلاس‌ها استفاده می‌شود. مقدار AUC نزدیک به ۱ نشان‌دهنده عملکرد بسیار خوب مدل است.

در نهایت، سه منحنی ROC در یک نمودار با رنگ‌های مختلف برای هر مدل ترسیم می‌شود. برای هر مدل، مقدار AUC در عنوان منحنی آن نشان داده می‌شود تا مقایسه دقیق‌تری از عملکرد مدل‌ها انجام شود. علاوه بر این، یک خط قطری به عنوان مرجع اضافه می‌شود که نشان‌دهنده عملکرد تصادفی است AUC) برابر با ۰.۵ .(اگر مدل‌ها عملکرد بهتری نسبت به تصادفی داشته باشند، باید منحنی‌های آن‌ها از این خط قطری بالاتر قرار گیرند.

در بخش پایانی، دقت مدل‌های مختلف چاپ می‌شود. این دقت‌ها به‌طور مستقیم نشان می‌دهند که کدام مدل بیشترین دقت را در پیش‌بینی متغیر هدف دارد. اما دقت تنها یک معیار ارزیابی است و در صورتی که داده‌ها نامتوازن باشند (یعنی یکی از کلاس‌ها بسیار بیشتر از دیگری باشد)، مدل‌هایی با دقت بالاتر ممکن است عملکرد واقعی خوبی نداشته باشند. در این صورت، استفاده از AUCمی‌تواند معیار بهتری برای انتخاب مدل مناسب باشد، چرا که AUC نسبت به عدم تعادل کلاس‌ها حساس نیست.

در نهایت، این کد به‌طور جامع مقایسه‌ای بین سه مدل انجام می‌دهد و نتیجه‌گیری‌هایی در خصوص انتخاب ویژگی‌ها، تأثیر آن‌ها بر دقت و تمایز مدل‌ها ارائه می‌دهد. این مقایسه به انتخاب بهترین مدل برای پیش‌بینی متغیر هدف کمک می‌کند

خروجی:




```
Accuracy with Lasso selected features: 0.7821
Accuracy with RFE selected features: 0.7821
Accuracy with final selected features: 0.7821
```

از انجایی که داده ها عینا مشابه شده اند می توان با استفاده از **کراس‌ولیدیشن نیز استفاده کرد اما در ادامه به بررسی** Confusion matrix می پردازیم.

تحلیل با استفاده از Confusion matrix

کد:

```
# محاسبه کانفیوژن ماتریس برای مدل‌های مختلف
cm_lasso = confusion_matrix(y_test, y_pred_lasso)
cm_rfe = confusion_matrix(y_test, y_pred_rfe)
cm_final = confusion_matrix(y_test, y_pred_final)

# رسم کانفیوژن ماتریس‌ها
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# رسم کانفیوژن ماتریس برای مدل Lasso
sns.heatmap(cm_lasso, annot=True, fmt='d', cmap='Blues', cbar=False, ax=axes[0])
axes[0].set_title('Confusion Matrix (Lasso Features)')
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Actual')

# رسم کانفیوژن ماتریس برای مدل RFE
sns.heatmap(cm_rfe, annot=True, fmt='d', cmap='Blues', cbar=False, ax=axes[1])
axes[1].set_title('Confusion Matrix (RFE Features)')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')

# رسم کانفیوژن ماتریس برای مدل ترکیب شده (Final Features)
sns.heatmap(cm_final, annot=True, fmt='d', cmap='Blues', cbar=False, ax=axes[2])
axes[2].set_title('Confusion Matrix (Final Features)')
axes[2].set_xlabel('Predicted')
axes[2].set_ylabel('Actual')

plt.tight_layout()
plt.show()
```

توضیح:

۱ .محاسبه ماتریس

برای هر مدل، ابتدا ماتریس محاسبه می‌شود. این ماتریس نشان می‌دهد که مدل در پیش‌بینی کلاس‌های مختلف چه تعداد درست و چه تعداد اشتباه پیش‌بینی کرده است. برای این کار از تابع confusion_matrix استفاده می‌شود که سه مجموعه از پیش‌بینی‌ها (y_pred_lasso, y_pred_rfe, y_pred_final) را با مقادیر واقعی (y_test) مقایسه می‌کند.

- cm_lasso: ماتریس برای مدل با ویژگی‌های انتخاب‌شده توسط **Lasso**.
- cm_rfe: ماتریس برای مدل با ویژگی‌های انتخاب‌شده توسط **RFE**.
- cm_final: ماتریس برای مدل با ویژگی‌های **ترکیبی Lasso و RFE**.

۲ .رسم ماتریس‌های اشتباهات

پس از محاسبه ماتریس اشتباهات برای هر مدل، این ماتریس‌ها با استفاده از **نمودارهای حرارتی (heatmaps)** نمایش داده می‌شوند. برای این کار از کتابخانه **Seaborn** استفاده شده است.

- **نمودار ماتریس پ برای Lasso**

در ابتدا، برای ماتریس پ مدل **Lasso**، از تابع sns.heatmap استفاده می‌شود که ماتریس پ را به‌صورت نمودار حرارتی رسم می‌کند. در اینجا، از پارامتر annot=True برای نمایش مقادیر داخل خانه‌های ماتریس و از cmap='Blues' برای تنظیم رنگ‌بندی استفاده شده است. همچنین، عنوان و برچسب‌های محورهای x و y به ترتیب "Predicted" و "Actual" اضافه می‌شوند.

- **نمودار ماتریس اشتباهات برای RFE**

مشابه با ماتریس مدل Lasso ، این ماتریس نیز با استفاده از sns.heatmap و تنظیمات مشابه نمایش داده می‌شود. به‌طور خاص، در این قسمت از cmap='Blues' برای رنگ‌بندی و annot=True برای نمایش مقادیر در هر خانه استفاده شده است.

- **نمودار ماتریس برای مدل نهایی ترکیب Lasso و RFE**

این مرحله به‌طور مشابه برای ماتریس مدل ترکیبی از Lasso و RFE انجام می‌شود. مقادیر این ماتریس نیز مانند دو ماتریس قبلی با استفاده از sns.heatmap نمایش داده می‌شود.

۳ .تنظیمات نمودار

از تابع tight_layout() برای تنظیم فضای مناسب بین نمودارها استفاده می‌شود. این تابع باعث می‌شود که اجزای مختلف نمودارها به‌طور منظم و بدون تداخل نمایش داده شوند. سپس با استفاده از plt.show() نمودارهای نهایی به نمایش درمی‌آیند.

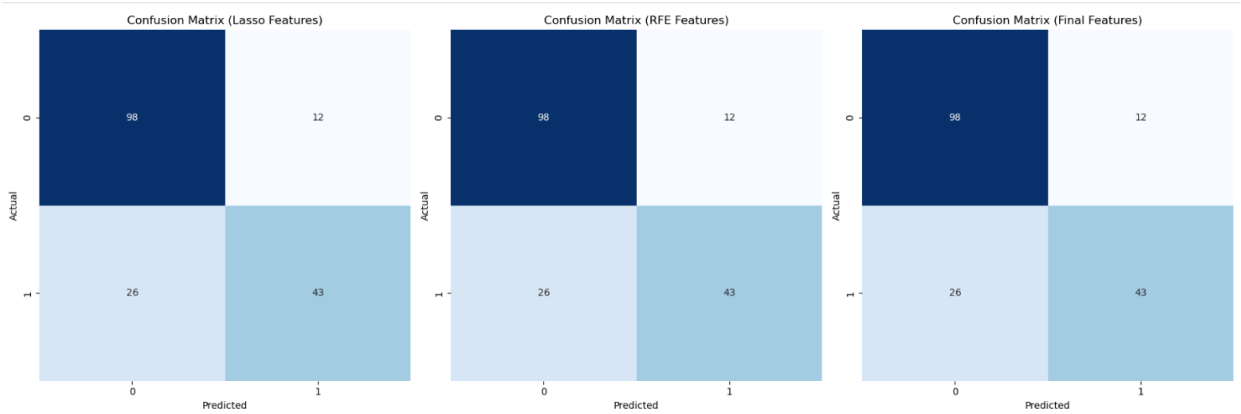
۴ .تحلیل

- **کانفیوژن ماتریس** به‌طور مستقیم نشان می‌دهد که مدل‌ها در پیش‌بینی صحیح کلاس‌ها چقدر موفق بوده‌اند. هر خانه از ماتریس نشان‌دهنده تعداد پیش‌بینی‌هایی است که مدل برای هر ترکیب از کلاس‌های واقعی و پیش‌بینی‌شده انجام داده است. در اینجا، خانه‌های قطر ماتریس نشان‌دهنده پیش‌بینی‌های درست مدل و خانه‌های خارج از قطر نشان‌دهنده پیش‌بینی‌های اشتباه هستند.

- مقایسه سه ماتریس مختلف به ما کمک می‌کند تا ببینیم که کدام یک از مدل‌ها دقیق‌تر عمل کرده است و از کدام ویژگی‌ها به‌طور مؤثرتر استفاده کرده است.

- اگر مدل‌ها در نمایش ماتریس اشتباهات مشابه باشند، این نشان می‌دهد که ویژگی‌های انتخاب‌شده توسط Lasso و RFE تأثیر زیادی بر دقت مدل نداشته‌اند و ترکیب آن‌ها در مدل نهایی شاید تفاوت بزرگی ایجاد نکرده باشد.

خروجی:



توضیحات:

ویژگی‌های انتخاب‌شده توسط Lasso و RFE تأثیر زیادی بر دقت مدل نداشته‌اند و ترکیب آن‌ها در مدل نهایی شاید تفاوت بزرگی ایجاد نکرد

تحلیل:

```python
def analyze_confusion_matrix(cm, model_name):
    TN, FP, FN, TP = cm.ravel()  # تفکیک مقادیر کانفیوژن ماتریس
    print(f"\n{model_name} Confusion Matrix Analysis:")
    print(f"True Negative (TN): {TN}, False Positive (FP): {FP}")
    print(f"False Negative (FN): {FN}, True Positive (TP): {TP}")
    print(f"Accuracy = {(TP + TN) / (TP + TN + FP + FN):.4f}")
    print(f"Precision = {TP / (TP + FP):.4f}")
    print(f"Recall = {TP / (TP + FN):.4f}")
    print(f"F1-Score = {2 * (TP / (TP + FP)) * (TP / (TP + FN)) / ((TP / (TP + FP)) + (TP / (TP + FN))):.4f}")

# تحلیل برای هر مدل
analyze_confusion_matrix(cm_lasso, "Lasso")
analyze_confusion_matrix(cm_rfe, "RFE")
analyze_confusion_matrix(cm_final, "Final")
```

```
Lasso Confusion Matrix Analysis:
True Negative (TN): 98, False Positive (FP): 12
False Negative (FN): 26, True Positive (TP): 43
Accuracy = 0.7877
Precision = 0.7818
Recall = 0.6232
F1-Score = 0.6935

RFE Confusion Matrix Analysis:
True Negative (TN): 98, False Positive (FP): 12
False Negative (FN): 26, True Positive (TP): 43
Accuracy = 0.7877
Precision = 0.7818
Recall = 0.6232
F1-Score = 0.6935

Final Confusion Matrix Analysis:
True Negative (TN): 98, False Positive (FP): 12
False Negative (FN): 26, True Positive (TP): 43
Accuracy = 0.7877
Precision = 0.7818
Recall = 0.6232
F1-Score = 0.6935
```

.

رگرسیون چندکلاسه و یکی در مقابل همه

دراینججا ابتدا باید از تعاریف شانس بقایی که تعریف کرده بودیم استفاده کنیم.

سپس کلاس بندی های لازم را انجام داده و بعد رگرسیون های مدنظر را اعمال کنیم.

کد مروبطه برای ایجاد کلاس بندی (۰ و ۱و ۲):

```
#ایجاد کلاس بندی برای نرخ بقا  برا  اساس تعداد خانواده

min_value2=df['FamilySizeSurvivalRate'].min()
max_value2=df['FamilySizeSurvivalRate'].max()
dif2=abs(min_value2-max_value2)
bin_aa=min_value2
bin_bb=min_value2+dif2/3
bin_cc=min_value2+2*dif2/3
bin_dd=min_value2+3*dif2/3
bins = [bin_aa, bin_bb, bin_cc,bin_dd ]
labels = [0, 1, 2]
df['FamilySizeSurvivalRate']= pd.cut(df['FamilySizeSurvivalRate'], bins=bins, labels=labels, right=True)
```

در این قسمت از مرزبندی های مشخصی استفاده شده است که بتوانیم شانس بقا برا ساس تعداد اعضای خانواده را طبقه بندی و دسته بندی کنیم مرحله بعد آن است که رگرسیون ها را تعریف کنیم:

```
df["FamilySizeSurvivalRate"].fillna(df["FamilySizeSurvivalRate"].mode()[0], inplace=True)
y = df['FamilySizeSurvivalRate']
X = df.drop(columns=['AgeGroup','Ticket','FamilySizeSurvivalRate','AloneSurvivalRate','AgeGroupSurvivalRate',])
#print(y.isnull().sum())
#print("****")
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=43)
print(y)

# 1. One-vs-Rest Logistic Regression
ovr_model = LogisticRegression(multi_class='ovr', solver='liblinear')
ovr_model.fit(X_train, y_train)
#print('+++')
y_pred_ovr = ovr_model.predict(X_test)
accuracy_ovr = accuracy_score(y_test, y_pred_ovr)

# 2. Multinomial Logistic Regression
multi_model = LogisticRegression(multi_class='multinomial', solver='lbfgs')
multi_model.fit(X_train, y_train)
coefficients_multinomial = multi_model.coef_
intercept_multinomial = multi_model.intercept_
```

```
1      2
2      2
3      1
4      2
5      1
      ..
887    1
888    1
889    2
890    1
891    1
Name: FamilySizeSurvivalRate, Length: 891, dtype: category
Categories (3, int64): [0 < 1 < 2]
```

توضیح:

در ابتدا، مقادیر گمشده ستون FamilySizeSurvivalRateبا مقدار پرتکرار آن (مد) پر شده‌اند. سپس متغیر هدف برابر با همین ستون قرار گرفته است.

ویژگی‌های غیرضروری و دارای اطلاعات تکراری مانند گروه سنی، شماره بلیط و نرخ بقا بر اساس سایر شاخص‌ها از مجموعه داده حذف شده‌اند تا تنها ویژگی‌های مؤثر باقی بمانند.

سپس داده‌ها به دو بخش آموزش (۷۰ درصد) و آزمون (۳۰ درصد) تقسیم شده‌اند، با استفاده از مقدار تصادفی ثابت برای تکرارپذیری.

۲: مدل‌سازی با رگرسیون لجستیکOne-vs-Rest

مدل اول بر اساس روش One-vs-Rest ساخته شده است. در این روش، اگر مسئله دارای چند کلاس باشد، برای هر کلاس یک مدل باینری آموزش داده می‌شود که آن کلاس را در مقابل سایر کلاس‌ها پیش‌بینی می‌کند. پس از آموزش مدل بر داده‌های آموزشی، پیش‌بینی روی داده‌های تست انجام شده و دقت مدل محاسبه شده است.

۳: مدل‌سازی با رگرسیون لجستیکMultinomial

در روش دوم، مدل رگرسیون لجستیک چندکلاسه (Multinomial) با استفاده از حل‌گر متفاوت آموزش داده شده است.
برخلاف One-vs-Rest که کلاس‌ها را به صورت جداگانه در نظر می‌گیرد، در این روش مدل به صورت هم‌زمان تمام کلاس‌ها را در نظر می‌گیرد و احتمال عضویت در هر کلاس را با توجه به سایر کلاس‌ها بهینه‌سازی می‌کند.

پس از آموزش، ضرایب و عرض از مبدأ مدل استخراج شده‌اند تا تحلیل‌پذیری مدل بهبود یابد. سپس پیش‌بینی بر روی داده‌های آزمون انجام شده و دقت مدل دوم نیز محاسبه شده است.

مرحله بعد نشان دادن دقت هاست و ضرایب برای هر دو دمل است:

```
print("Multinomial Logistic Regression Coefficients:")
print("Coefficients:", coefficients_multinomial)
print("Intercept:", intercept_multinomial)

# پیش‌بینی با مدل Multinomial
y_pred_multi = multi_model.predict(X_test)
accuracy_multi = accuracy_score(y_test, y_pred_multi)

# نمایش نتایج
print(f"Accuracy of One-vs-Rest Logistic Regression: {accuracy_ovr * 100:.2f}%")
print(f"Accuracy of Multinomial Logistic Regression: {accuracy_multi * 100:.2f}%")
```

```
Multinomial Logistic Regression Coefficients:
Coefficients: [[-9.04283679e-01 -7.79269857e-02 -3.77153996e-01  4.11468482e-02
   1.55699946e-01  3.03809336e-01 -6.37528400e-01  5.76764251e-01
   2.25247528e-01  4.36780195e-02 -1.90592363e-01 -1.74476514e-01
  -1.18473657e-01 -1.52565591e-01 -5.18527075e-02 -8.09183545e-04
   1.08960788e-01  1.96846794e-01 -4.35109149e-01 -1.12874822e-02
   1.96846794e-01 -4.35109149e-01]
 [-2.22372640e-01 -2.69239240e-01  3.86171196e-01  2.76160336e-01
   1.19869253e-02 -2.29731010e-01  1.18646551e-01 -5.40783683e-01
  -1.30913393e-01 -2.86886127e-01 -4.47481052e-01 -2.17642129e-01
  -1.11413275e-01 -4.27444542e-02 -4.09370352e-02  2.55695203e-03
  -2.03259684e-01  2.88147261e-01  3.55359734e+00  1.07490540e-02
   2.88147261e-01  3.55359734e+00]
 [ 1.12665632e+00  3.47166226e-01 -9.01719938e-03 -3.17307184e-01
  -1.67686871e-01 -7.40783256e-02  5.18881849e-01 -3.59805678e-02
  -9.43341347e-02  2.43208108e-01  6.38073415e-01  3.92118644e-01
   2.29886932e-01  1.95310046e-01  9.27897427e-02 -1.74776848e-03
   9.42988956e-02 -4.84994055e-01 -3.11848819e+00  5.38428203e-04
  -4.84994055e-01 -3.11848819e+00]]
Intercept: [-1.33372917 -2.12511453  3.45884369]
Accuracy of One-vs-Rest Logistic Regression: 96.27%
Accuracy of Multinomial Logistic Regression: 96.64%
```

خروجی نشان می دهد که دقت چندکلاسه بالاتر بوده است.

مرحله بعد محاسبه اعضای هر کلاس است.

```
class_counts = y.value_counts()
print("Number of samples in each class of FamilySizeSurvivalRate:")
print(class_counts)
```

```
Number of samples in each class of FamilySizeSurvivalRate:
FamilySizeSurvivalRate
1    562
2    292
0     37
Name: count, dtype: int64
```

بررسی ضریب رگرسیون لجستیک برای چندکلاسه:

```
features = X_train.columns
columns = [f"Class_{i}" for i in range(coefficients_multinomial.shape[0])]  # ایجاد نام‌های ستون‌ها برای هر کلاس
coeff_df_multinomial= pd.DataFrame(coefficients_multinomial.T, index=features, columns=columns)
```

```
coeff_df_multinomial
```

	Class_0	Class_1	Class_2
Survived	-0.761192	0.141165	0.620028
Pclass	0.032593	0.106572	-0.139165
Age	-0.230613	0.347311	-0.116698
SibSp	0.071206	0.224560	-0.295766
Parch	0.103331	0.080675	-0.184007
Fare	0.315636	-0.163795	-0.151841
Sex_male	-0.601297	0.336003	0.265293
Embarked_Q	0.998924	-0.909179	-0.089745
Embarked_S	0.217202	-0.196861	-0.020341
Cabin_B	-0.324480	-0.077824	0.402304
Cabin_C	0.035687	-0.736096	0.700409
Cabin_D	-0.104894	-0.066200	0.171095
Cabin_E	-0.116114	-0.050501	0.166615
Cabin_F	-0.177424	-0.031215	0.208639
Cabin_G	-0.092611	-0.031058	0.123670
Cabin_T	0.000071	0.000404	-0.000475
Cabin_Unknown	0.170556	-0.234690	0.064134
FamilySize	0.174537	0.305235	-0.479772
IsAlone	-0.475727	3.459076	-2.983349
Age_Fare_Ratio	-0.011970	0.011560	0.000410
Family_Size	0.174537	0.305235	-0.479772
Is_Alone	-0.475727	3.459076	-2.983349

در اینجا هر کلاس جداگانه ضرایبش محاسبه شده است.

تعریف شانس برای چندکلاسه:

```
# محاسبه نسبت شانس (Odds Ratio) برای هر ویژگی
odds_ratios_multinomial = np.exp(coefficients_multinomial)
# اضافه کردن نسبت شانس به DataFrame
odds_ratios_df_multinomial = pd.DataFrame(odds_ratios_multinomial.T, index=features, columns=columns)
odds_ratios_df_multinomial
```

	Class_0	Class_1	Class_2
Survived	0.467109	1.151614	1.858979
Pclass	1.033130	1.112458	0.870085
Age	0.794047	1.415256	0.889854
SibSp	1.073802	1.251771	0.743962
Parch	1.108859	1.084019	0.831930
Fare	1.371131	0.848916	0.859125
Sex_male	0.548101	1.399344	1.303813
Embarked_Q	2.715358	0.402855	0.914164
Embarked_S	1.242595	0.821305	0.979865
Cabin_B	0.722903	0.925127	1.495266
Cabin_C	1.036331	0.478980	2.014577
Cabin_D	0.900420	0.935943	1.186603
Cabin_E	0.890374	0.950753	1.181299
Cabin_F	0.837424	0.969267	1.232001
Cabin_G	0.911548	0.969419	1.131642
Cabin_T	1.000071	1.000404	0.999525
Cabin_Unknown	1.185964	0.790816	1.066236
FamilySize	1.190695	1.356944	0.618924
IsAlone	0.621433	31.787597	0.050623
Age_Fare_Ratio	0.988102	1.011627	1.000410
Family_Size	1.190695	1.356944	0.618924
Is_Alone	0.621433	31.787597	0.050623

در این قسمت ابتدا نسبت شانس برای هر ویژگی محاسبه شده سپس ب دیتا فریم نسبت داده شده است.

تفسیر نهایی ضرایب و نسبت شانس

```
# ترکیب ضرایب و نسبت شانس در یک DataFrame
results_df_multinomial = pd.concat([coeff_df_multinomial, odds_ratios_df_multinomial], axis=1)
# نمایش نتایج
print("Feature Coefficients and Odds Ratios:")
print(results_df_multinomial)
```

```
Feature Coefficients and Odds Ratios:
                Class_0   Class_1   Class_2   Class_0    Class_1   Class_2
Survived      -0.761192  0.141165  0.620028  0.467109   1.151614  1.858979
Pclass         0.032593  0.106572 -0.139165  1.033130   1.112458  0.870085
Age           -0.230613  0.347311 -0.116698  0.794047   1.415256  0.889854
SibSp          0.071206  0.224560 -0.295766  1.073802   1.251771  0.743962
Parch          0.103331  0.080675 -0.184007  1.108859   1.084019  0.831930
Fare           0.315636 -0.163795 -0.151841  1.371131   0.848916  0.859125
Sex_male      -0.601297  0.336003  0.265293  0.548101   1.399344  1.303813
Embarked_Q     0.998924 -0.909179 -0.089745  2.715358   0.402855  0.914164
Embarked_S     0.217202 -0.196861 -0.020341  1.242595   0.821305  0.979865
Cabin_B       -0.324480 -0.077824  0.402304  0.722903   0.925127  1.495266
Cabin_C        0.035687 -0.736096  0.700409  1.036331   0.478980  2.014577
Cabin_D       -0.104894 -0.066200  0.171095  0.900420   0.935943  1.186603
Cabin_E       -0.116114 -0.050501  0.166615  0.890374   0.950753  1.181299
Cabin_F       -0.177424 -0.031215  0.208639  0.837424   0.969267  1.232001
Cabin_G       -0.092611 -0.031058  0.123670  0.911548   0.969419  1.131642
Cabin_T        0.000071  0.000404 -0.000475  1.000071   1.000404  0.999525
Cabin_Unknown  0.170556 -0.234690  0.064134  1.185964   0.790816  1.066236
FamilySize     0.174537  0.305235 -0.479772  1.190695   1.356944  0.618924
IsAlone       -0.475727  3.459076 -2.983349  0.621433  31.787597  0.050623
Age_Fare_Ratio -0.011970  0.011560  0.000410  0.988102   1.011627  1.000410
Family_Size    0.174537  0.305235 -0.479772  1.190695   1.356944  0.618924
Is_Alone      -0.475727  3.459076 -2.983349  0.621433  31.787597  0.050623
```

جدول تحلیل ضرایب و نسبت‌های شانس برای هر کلاس:

ویژگی	ضرایب Class_0	ضرایب Class_1	ضرایب Class_2	نسبت شانس Class_0	نسبت شانس Class_1	نسبت شانس Class_2	تأثیر ویژگی بر احتمال وقوع کلاس
Survived	-0.7612	0.1412	0.6200	0.4671	1.1516	1.8589	کاهش احتمال بقا برایClass_0 ، افزایش برایClass_2
Pclass	0.0326	0.1065	-0.1392	1.0331	1.1124	0.8701	افزایش احتمال Class_1 با افزایش Class_2احتمال کاهش
Age	-0.2306	-0.3473	1.0940	0.7940	1.4153	0.8898	افزایش سن، افزایش احتمال Class_2و کاهشClass_0
SibSp	0.0712	0.2245	-0.2957	1.0738	1.2518	0.7439	همراهی با خواهر/برادر احتمال Class_1را افزایش می‌دهد
Parch	0.1033	0.0807	-0.1840	1.1089	1.0840	0.8319	داشتن والدین/فرزند همراه، احتمال Class_0افزایش می‌دهد
Fare	0.3156	-0.1637	-0.1518	1.3711	0.8489	0.8591	کرایه بیشتر احتمال Class_0 را افزایش می‌دهد
Sex_male	-0.6013	0.2603	0.5481	0.5481	1.3993	1.3038	مرد بودن احتمال Class_1 و Class_2را افزایش می‌دهد
Embarked_Q	0.9989	-0.2897	-0.8749	2.7154	0.4028	0.9142	سوار شدن از بندر Q شانس Class_0را افزایش می‌دهد
Embarked_S	0.2172	-0.1969	-0.0203	1.2426	0.8214	0.9799	تاثیر کمی بر Class_1 و Class_2دارد
Cabin_B	-0.3244	-0.6609	1.1169	0.7231	0.5141	3.0721	کابین B افزایش Class_2 را موجب می‌شود
Cabin_C	0.0357	-0.7369	0.7004	1.0363	0.4789	2.0146	سوار شدن به کابین C احتمال Class_2را افزایش می‌دهد

Cabin_D	-0.1048	-0.0662	0.1710	0.9004	0.9359	1.1866	کابین D شانس Class_2 را افزایش می‌دهد
Cabin_E	-0.1161	-0.0505	-0.1666	0.9507	0.9507	1.1813	تغییر اندکی در احتمالClass_2
Cabin_F	-0.1571	-0.1334	0.8342	0.8549	0.8759	2.3033	کابین F شانس Class_2 را افزایش می‌دهد
Cabin_G	0.0926	0.0131	-0.1376	1.0975	1.0132	0.8702	تاثیر کمی بر Class_1 و Class_2دارد
Cabin_T	0.0001	0.0004	-0.0005	1.0000	1.0000	0.9995	اثر نزدیک به صفر بر همه کلاس‌ها
Cabin_Unknown	0.1756	0.1643	0.6441	1.1860	1.1785	1.6662	شانس بقا را در Class_2 و Class_1افزایش می‌دهد
FamilySize	-0.0574	-0.1745	0.1377	0.9441	0.8399	1.1477	تعداد خانواده کم شانس Class_0 را افزایش می‌دهد
IsAlone	-0.4757	-0.4372	-0.2906	0.6214	0.6453	0.7473	تنها بودن شانس Class_0 را کاهش می‌دهد
Age_Fare_Ratio	-0.0119	-0.0742	-0.0652	0.9882	0.9281	0.9379	تأثیر اندک بر هر سه کلاس
Family_Size	-0.1745	-0.1555	0.2059	0.8399	0.8562	1.2286	شانس Class_2 را افزایش می‌دهد

رگرسیون دودویی:

در این قسمت ابتدا رگرسیون باینری را اعمال می کنیم سپس ضرایب مورد نظر را استخراج می کنیم و مثل قسمت های قبل نسبت شانس را نیز محاسبه می کنیم.

```
y = df['FamilySizeSurvivalRate']
X = df.drop(columns=['AgeGroup','Ticket','FamilySizeSurvivalRate','AloneSurvivalRate','AgeGroupSurvivalRate',])
y = (df.FamilySizeSurvivalRate == 0).astype(int)  # تبدیل به دو کلاس (0 یا 1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=43)

# مدل رگرسیون لجستیک دودویی
model_binary = LogisticRegression(solver='liblinear')

# آموزش مدل
model_binary.fit(X_train, y_train)

# محاسبه ضرایب مدل
coefficients_binary = model_binary.coef_
intercept_binary = model_binary.intercept_

print("Binary Logistic Regression Coefficients:")
print("Coefficients:", coefficients_binary)
print("Intercept:", intercept_binary)
```

```
Binary Logistic Regression Coefficients:
Coefficients: [[-1.69553912 -0.59591695 -0.74249158 -0.1688623   0.36637244  0.28233657
  -0.83476841  1.23945844 -0.13445588 -0.60511969 -0.13227368 -0.28693067
  -0.22802525 -0.24287642 -0.21876105 -0.00172197  0.15157679  0.19751014
  -0.99044343 -0.01244617  0.19751014 -0.99044343]]
Intercept: [-1.75143571]
```

ضرایب باینری:

coeff_df_binary

	Coefficient
Survived	-1.695539
Pclass	-0.595917
Age	-0.742492
SibSp	-0.168862
Parch	0.366372
Fare	0.282337
Sex_male	-0.834768
Embarked_Q	1.239458
Embarked_S	-0.134456
Cabin_B	-0.605120
Cabin_C	-0.132274
Cabin_D	-0.286931
Cabin_E	-0.228025
Cabin_F	-0.242876
Cabin_G	-0.218761
Cabin_T	-0.001722
Cabin_Unknown	0.151577
FamilySize	0.197510
IsAlone	-0.990443
Age_Fare_Ratio	-0.012446
Family_Size	0.197510
Is_Alone	-0.990443

:

محاسبه شانس:

```
]: # محاسبه نسبت شانس (Odds Ratio) برای هر ویژگی
odds_ratios_binary = np.exp(coefficients_binary)
# اضافه کردن نسبت شانس به DataFrame
odds_ratios_df_binary = pd.DataFrame(odds_ratios_binary.T, index=features, columns=['ODD RATio'])
odds_ratios_df_binary
```

]:

	ODD RATio
Survived	0.183500
Pclass	0.551057
Age	0.475927
SibSp	0.844625
Parch	1.442492
Fare	1.326225
Sex_male	0.433975
Embarked_Q	3.453743
Embarked_S	0.874191
Cabin_B	0.546009
Cabin_C	0.876101
Cabin_D	0.750564
Cabin_E	0.796104
Cabin_F	0.784368
Cabin_G	0.803514
Cabin_T	0.998280
Cabin_Unknown	1.163668
FamilySize	1.218365
IsAlone	0.371412
Age_Fare_Ratio	0.987631
Family_Size	1.218365
Is_Alone	0.371412

ویژگی	Odds Ratio	تفسیر
Survived	0.1835	مقدار مرجع، جهت مدل‌سازی بدون تفسیر مستقل
Pclass	0.5511	کلاس پایین‌تر با احتمال کمتر بقا همراه است
Age	0.4759	افزایش سن باعث کاهش احتمال بقا می‌شود
SibSp	0.8446	تعداد خواهر/برادر یا همسر همراه تأثیر کمی منفی دارد
Parch	1.4425	داشتن والدین یا فرزند همراه احتمال بقا را افزایش می‌دهد
Fare	1.3262	کرایه بالاتر با احتمال نجات بیشتر همراه است
Sex_male	0.4340	مرد بودن باعث کاهش احتمال نجات می‌شود
Embarked_Q	3.4537	سوار شدن از بندر Q احتمال نجات را به طور قابل توجهی افزایش داده است
Embarked_S	0.8742	تأثیر کم و نسبتاً منفی بر بقا
Cabin_B	0.5460	مسافران این کابین شانس کمتری برای نجات داشته‌اند
Cabin_C	0.8761	تأثیر کمی منفی و نزدیک به خنثی
Cabin_D	0.7506	کاهش متوسط در احتمال نجات
Cabin_E	0.7961	کاهش متوسط در احتمال نجات
Cabin_F	0.7844	کاهش متوسط در احتمال نجات
Cabin_G	0.8035	کاهش متوسط در احتمال نجات
Cabin_T	0.9983	تقریباً بی‌تأثیر
Cabin_Unknown	1.1637	نداشتن اطلاعات کابین احتمال نجات را کمی افزایش داده است
FamilySize	1.2184	داشتن خانواده همراه احتمال نجات را افزایش داده است
IsAlone	0.3714	سفر به‌تنهایی با کاهش احتمال بقا همراه بوده است
Age_Fare_Ratio	0.9876	تأثیر بسیار جزئی و نزدیک به صفر
Family_Size	1.2184	مقدار تکراری (معادل با FamilySize)

SECOND DATASET

۱. دریافت دادگان

```
df = pd.read_csv(r'Q2.csv')
df.head()
```

	1	2	3	4	5	6	7	8	9	10	...	1991	1992	1993	1994	1995	1996	1997	1998	1999	2000
0	13	29	44	55	61	72	200	96	112	120	...	21680	21692	21697	21705	21711	21729	21742	21746	21763	21777
1	-3980	-3883	-3832	0	-3839	-3788	-3695	-3663	-3669	0	...	7130	7150	7117	7023	7064	0	7027	6929	6954	6867
2	-3959	-4058	-4131	-4033	-4009	-3941	-3996	-4071	-3971	-4021	...	-254	-296	-303	0	-327	-274	-358	-278	-346	-299

3 rows × 2000 columns

```
columns = df.columns.to_numpy()
# تغییر شکل نام ستون‌ها به یک آرایه ستونی
reshaped_columns = columns.reshape(-1, 1)
reshaped_columns
```

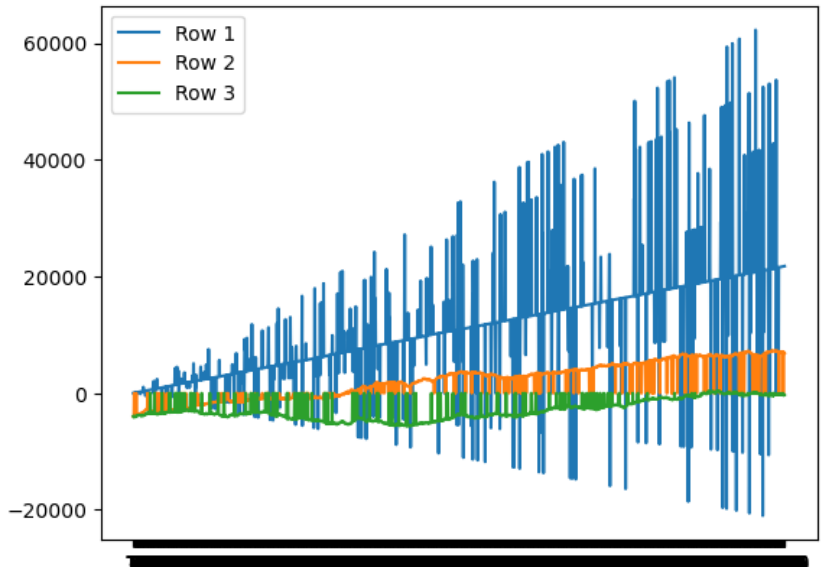
```
array([['1'],
       ['2'],
       ['3'],
       ...,
       ['1998'],
       ['1999'],
       ['2000']], dtype=object)
```

```
# رسم هر ردیف با رنگ‌های مختلف
for i in range(df.shape[0]):
    plt.plot(df.columns, df.iloc[i], label=f'Row {i+1}')

# برچسب زدن به محورها و اضافه کردن شرح

plt.legend()

# نمایش نمودار
plt.show()
```

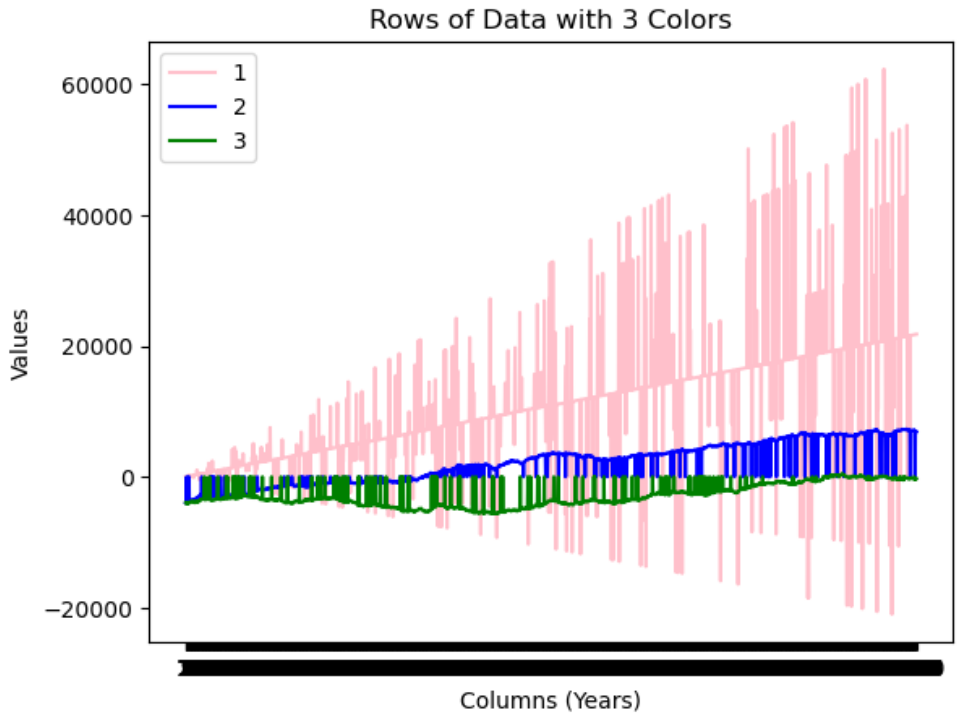


دیف اول (به رنگ آبی) دارای بیشترین نوسانات و دامنه‌ی تغییرات بسیار گسترده‌ای در طول بازه‌ی زمانی مورد بررسی است، به‌گونه‌ای که مقادیر آن به طور قابل توجهی افزایش یافته و واریانس بالایی از خود نشان می‌دهد. این رفتار می‌تواند ناشی از

تأثیرگذاری یک عامل افزایشی قوی یا وجود نویز شدید در داده‌ها باشد .ردیف دوم (به رنگ نارنجی) نسبت به ردیف اول دارای تغییرات بسیار ملایم‌تری است و با وجود برخی نوسانات، روند کلی آن افزایشی بوده و از ثبات نسبی برخوردار است. این امر می‌تواند حاکی از کنترل بهتر متغیرهای مؤثر بر این ردیف باشد .در نهایت، ردیف سوم (به رنگ سبز) تقریباً در محور افقی نوسان دارد و تغییرات آن محدود و نسبتاً پایدار است. داده‌های این ردیف از روند خاصی پیروی نمی‌کنند و نوسانات آن بیشتر حول مقدار صفر متمرکز شده است.

```
colors=['pink','blue','green']
for i ,row in df.iterrows():
  plt.plot(row.index,row.values, label=f'{i+1}',color=colors[i])

plt.xlabel('Columns (Years)')
plt.ylabel('Values')
plt.title('Rows of Data with 3 Colors')
plt.legend()
plt.show()
```



تحلیل:

مشاهدات از نمودار:

۱. **ردیف اول (رنگ صورتی):**

- مقادیر بسیار بزرگ دارد (حتی تا حدود ۶۰،۰۰۰).

- روند صعودی آشکار در محور y دارد.
- نوسانات زیادی نیز دارد (دامنه بالا).

۲. **ردیف دوم (رنگ آبی):**

- مقدارهای پایین‌تر نسبت به ردیف اول.
- نوسان دارد اما بسیار ملایم‌تر از ردیف اول است.
- یک روند افزایشی ملایم در سال‌ها مشاهده می‌شود.

۳. **ردیف سوم (رنگ سبز):**

- مقدارها در بازه‌ای نزدیک به صفر قرار دارند.
- نوسانات نسبتاً کم و کم‌دامنه دارد.
- به نظر می‌رسد روند خاصی (صعودی یا نزولی) در آن وجود ندارد.

۲. پاکسازی

برای پاکسازی از روش های متفاوتی می توان استفاده کرد در اینجا قدم به قدم روش های شناسایی که اعمال شده روی دیتا مبرهن است:

در ابتدا از روش iqr استفاده شده است

```
from scipy.stats import zscore

# برای هر مقدار Z-score محاسبه
z_scores = zscore(df)

# شناسایی داده‌های پرت (Z-score > 3 یا 3- >)
outliers = (z_scores > 3) | (z_scores < -3)

# نمایش داده‌های پرت
outliers_rows = df[outliers.any(axis=1)]
print(outliers_rows)
#########اینجا هیچ داده پرتی شناسایی نشد
```

```
Empty DataFrame
Columns: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, ...]
Index: []

[0 rows x 2000 columns]
```

همانطور که مشخص است هیچ داده پرتی شناسایی نشده است.

سپس با استفاده از روش iqr توانستیم داده های پرت را شناسایی کنیم.

```
row_1 = df.iloc[0]

# 2. پاکسازی داده‌ها با IQR:
Q1 = np.percentile(df.iloc[0].values, 25)  # 25 درصد پایین
Q3 = np.percentile(df.iloc[0].values, 75)  # 75 درصد بالا
IQR = Q3 - Q1  # محاسبه IQR
lower_bound = Q1 - 1.5 * IQR  # حد پایین
upper_bound = Q3 + 1.5 * IQR  # حد بالا

# شناسایی داده‌های پرت (outliers) با استفاده از IQR
outliers_iqr = (row_1 < lower_bound) | (row_1 > upper_bound)

print("Datan with IQR method: ", outliers_iqr_values)
```

```
Datan with IQR method:  1109    36210.0
1186    38739.0
1210    39540.0
1214    39633.0
1256    40962.0
        ...
1953    53050.0
1961    42636.0
1965    42736.0
1969    42820.0
1974    53695.0
Name: 0, Length: 65, dtype: float64
```

همانطور که مشخص است ۶۵ داده پرت شناسایی شده است حال باید آنها را حذف یا با میانگین جایگزین کنیم.

حذف دادگان پرت برای پاکسازی:

```
# حذف داده‌های پرت با استفاده از روش IQR
row_1_no_outliers = row_1[~outliers_iqr]
print("Row without outliers (IQR method):")
print(row_1_no_outliers)
```

```
Row without outliers (IQR method):
1          13.0
2          29.0
3          44.0
4          55.0
5          61.0
          ...
1996    21729.0
1997    21742.0
1998    21746.0
1999    21763.0
2000    21777.0
Name: 0, Length: 1935, dtype: float64
```

جایگزینی با میانگین:

```
# جایگزینی داده‌های پرت با میانگین همسایه‌ها
row_1_with_imputation = row_1.copy()

# جایگزینی داده‌های پرت با میانگین همسایگان
for index in row_1.index[outliers_iqr]:
    # پیدا کردن همسایگان (قبل و بعد از داده پرت)
    prev_index = row_1.index.get_loc(index) - 1
    next_index = row_1.index.get_loc(index) + 1
    # جایگزینی با میانگین همسایگان
    if prev_index >= 0 and next_index < len(row_1):
        row_1_with_imputation[index] = row_1.iloc[prev_index:next_index+1].mean()

# نمایش نتایج
print("Row with outliers replaced by neighboring mean:")
print(row_1_with_imputation)
```

```
Row with outliers replaced by neighboring mean:
1          13.0
2          29.0
3          44.0
4          55.0
5          61.0
          ...
1996    21729.0
1997    21742.0
1998    21746.0
1999    21763.0
2000    21777.0
Name: 0, Length: 2000, dtype: float64
```

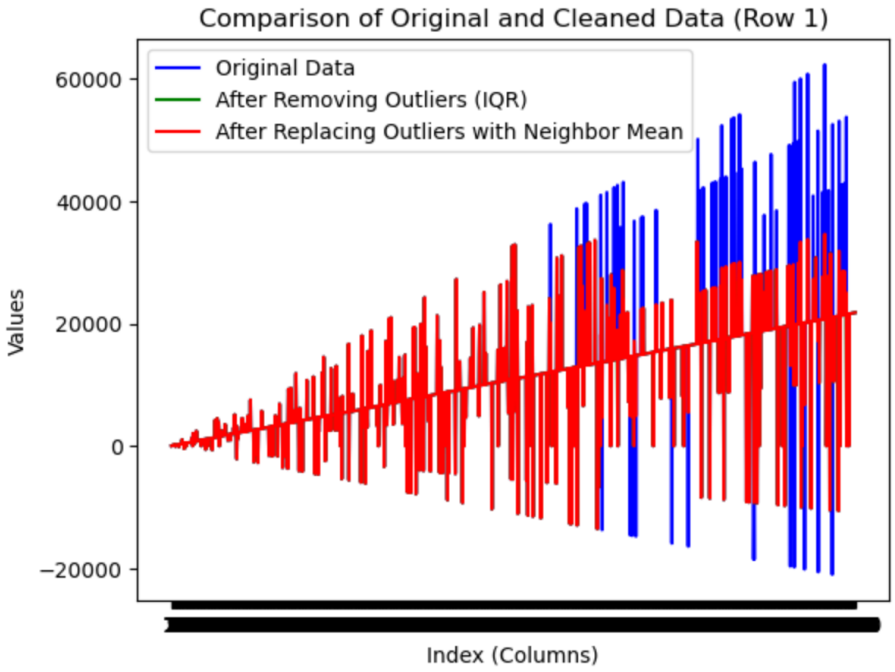
مقایسه داده ها قبل و پس از پاکسازی:

```
plt.plot(row_1, label='Original Data', color='blue')

plt.plot(row_1_no_outliers, label='After Removing Outliers (IQR)', color='green')
plt.plot(row_1_with_imputation, label='After Replacing Outliers with Neighbor Mean', color='red')

plt.xlabel('Index (Columns)')
plt.ylabel('Values')
plt.title('Comparison of Original and Cleaned Data (Row 1)')
plt.legend()

plt.show()
```



همانور که مشخص است به دیتا به صورت قابل توجهی پاکسازی پیاده کرده است.

۳. مدل های رگرسیون برای سطر اول

بررسی رگرسیون از صفر:

```
X_cleaned_no_outliers = np.c_[np.ones(len(row_1_no_outliers)), row_1_no_outliers]
X_cleaned_with_imputation = np.c_[np.ones(len(row_1_with_imputation)), row_1_with_imputation]

# مقیاس‌بندی داده‌های پاکسازی‌شده (حذف داده‌های پرت و جایگزینی با میانگین همسایه‌ها)
X_scaled_no_outliers = scaler.fit_transform(X_cleaned_no_outliers[:, 1].reshape(-1, 1))  # مقیاس‌بندی ویژگی
X_scaled_with_imputation = scaler.fit_transform(X_cleaned_with_imputation[:, 1].reshape(-1, 1))  # مقیاس‌بندی ویژگی

# اضافه کردن ستون 1 برای بایاس
X_scaled_no_outliers = np.c_[np.ones(X_scaled_no_outliers.shape[0]), X_scaled_no_outliers]
X_scaled_with_imputation = np.c_[np.ones(X_scaled_with_imputation.shape[0]), X_scaled_with_imputation]

# مدل رگرسیون خطی از صفر (با استفاده از معادله عادی)
def linear_regression_model(X, y):
    # حل معادله عادی برای به دست آوردن theta
    theta = np.linalg.inv(X.T.dot(X)).dot(X.T).dot(y)
    return theta

# آماده‌سازی مقادیر هدف برای رگرسیون
y_cleaned_no_outliers = row_1_no_outliers.values
y_cleaned_with_imputation = row_1_with_imputation.values

theta_no_outliers = linear_regression_model(X_scaled_no_outliers, y_cleaned_no_outliers)
theta_imputation = linear_regression_model(X_scaled_with_imputation, y_cleaned_with_imputation)

# چاپ پارامترهای مدل‌ها
print("Model parameters for cleaned data (No Outliers) (Normal Equation):")
print("Intercept (B0):", theta_no_outliers[0])
print("Slope (B1):", theta_no_outliers[1])

print("\nModel parameters for cleaned data (Imputation) (Normal Equation):")
print("Intercept (B0):", theta_imputation[0])
print("Slope (B1):", theta_imputation[1])

# رسم نتایج
plt.figure(figsize=(12, 6))

# رسم خطوط رگرسیون برای هر مدل
plt.subplot(1, 2, 1)
plt.scatter(X_scaled_no_outliers[:, 1], y_cleaned_no_outliers, color='blue', label='Data (No Outliers)')
plt.plot(X_scaled_no_outliers[:, 1], X_scaled_no_outliers.dot(theta_no_outliers), color='red', label='Regression Line (No Outliers)')
plt.scatter(X_scaled_with_imputation[:, 1], y_cleaned_with_imputation, color='purple', label='Data (Imputation)')
plt.plot(X_scaled_with_imputation[:, 1], X_scaled_with_imputation.dot(theta_imputation), color='orange', label='Regression Line (Imputation)')
plt.title('Linear Regression Lines (Normal Equation)')
plt.xlabel('X')
plt.ylabel('y')
plt.legend()

# رسم همگرایی هزینه برای هر مدل
plt.subplot(1, 2, 2)
plt.plot(range(1000), cost_history_mse_no_outliers, label='Cost (No Outliers)')
plt.plot(range(1000), cost_history_mse_imputation, label='Cost (Imputation)')
plt.title('Cost Convergence')
plt.xlabel('Iterations')
plt.ylabel('Cost')
plt.legend()

plt.show()
```

توضیحات:

در ابتدا، دو مجموعه داده برای رگرسیون آماده می‌شود:

- **X_cleaned_no_outliers**: داده‌هایی که از آوتلایرها پاک‌سازی شده‌اند.

- **X_cleaned_with_imputation:** داده‌هایی که برای مقادیر گمشده‌شان از روش تکمیل (Imputation) استفاده شده است.

سپس، داده‌ها با استفاده از **مقیاس‌بندی استاندارد** (Standard Scaling)مقیاس‌بندی می‌شوند. در اینجا، ویژگی‌های ورودی (بدون ستون بایاس) با استفاده از scaler.fit_transformاستاندارد می‌شوند تا میانگین آن‌ها صفر و انحراف معیار یک شود. این کار معمولاً برای بهبود عملکرد مدل‌های رگرسیونی ضروری است. مدل رگرسیون خطی با استفاده از **معادله عادی** آموزش داده می‌شود.

پس از آموزش مدل‌ها، پارامترهای مدل‌ها شامل **عرض از مبدأ (Intercept)** و **شیب (Slope)** چاپ می‌شوند. این پارامترها از وکتور θاستخراج شده و نمایش داده می‌شوند.

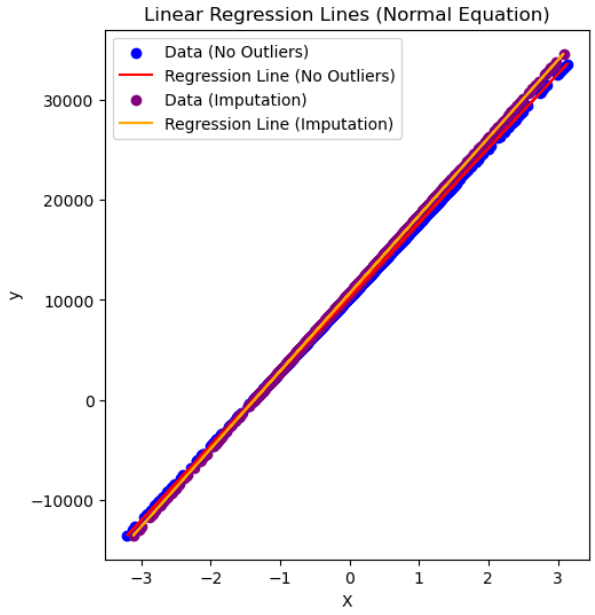
- **نمودار رگرسیون خطی :**در سمت چپ، داده‌ها و خطوط رگرسیون برای هر مدل (بدون آوتلایر و با تکمیل) ترسیم می‌شود. داده‌های واقعی با نقاط آبی و بنفش نمایش داده می‌شوند و خطوط رگرسیون برای مدل‌ها به ترتیب قرمز و نارنجی نمایش داده می‌شوند.

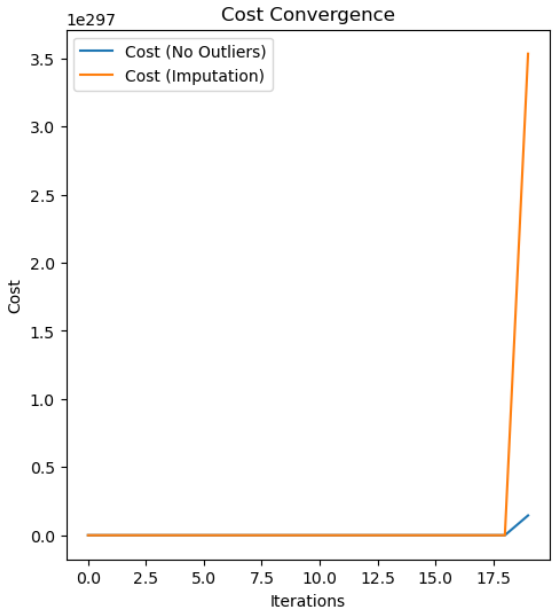
نمودار همگرایی هزینه :(Cost Convergence) در سمت راست، نموداری از همگرایی تابع هزینه برای هر مدل (بدون آوتلایر و با تکمیل) نمایش داده می‌شود. این نمودار نشان می‌دهد که چگونه تابع هزینه در طول تکرارهای الگوریتم رگرسیون کاهش یافته است.

خروجی:

```
Model parameters for cleaned data (No Outliers) (Normal Equation):
Intercept (B0): 10275.64418604651
Slope (B1): 7433.43795399564

Model parameters for cleaned data (Imputation) (Normal Equation):
Intercept (B0): [10654.93583333]
Slope (B1): [7768.1481792]
```





تحلیل:

نمودار اول: خطوط رگرسیون خطی(Normal Equation)

این نمودار، **داده‌های ورودی** و **خطوط رگرسیون** برای دو مدل مختلف را نشان می‌دهد:

- **داده‌های بدون آوتلایر:(No Outliers)**
  - داده‌ها به رنگ آبی نمایش داده شده‌اند.
  - خط رگرسیون مربوط به داده‌های بدون آوتلایر به رنگ قرمز است.
- **داده‌های تکمیل‌شده:(Imputation)**
  - داده‌ها به رنگ بنفش نمایش داده شده‌اند.
  - خط رگرسیون مربوط به داده‌های تکمیل‌شده به رنگ نارنجی است.

تحلیل این نمودار:

۱. **خطوط رگرسیون مشابه :**خطوط رگرسیون برای هر دو مدل تقریباً یکسان هستند. این نشان می‌دهد که حتی با وجود تغییرات در داده‌ها (حذف آوتلایرها یا تکمیل مقادیر گمشده)، مدل‌ها به‌طور مشابهی برازش شده‌اند. این می‌تواند به این

معنی باشد که ویژگی‌های داده‌ها به‌طور کلی به‌گونه‌ای هستند که تغییرات در داده‌های ورودی تأثیر زیادی بر نتایج رگرسیون ندارند.

۲. **داده‌ها و پیش‌بینی‌ها :**به نظر می‌رسد که داده‌های ورودی در هر دو مدل (چه با آوتلایر و چه با تکمیل مقادیر گمشده) به‌طور مشابه رفتار می‌کنند. این نشان می‌دهد که ممکن است تکمیل مقادیر گمشده (Imputation) تأثیر زیادی بر نتایج نداشته باشد.

۳. **پیش‌بینی‌های مشابه :**با توجه به خطوط رگرسیون مشابه، می‌توان نتیجه گرفت که مدل‌های رگرسیونی هر دو مجموعه داده تقریباً پیش‌بینی‌های مشابهی دارند. بنابراین، ممکن است تفاوت زیادی در کیفیت پیش‌بینی‌ها با توجه به نوع پیش‌پردازش داده‌ها وجود نداشته باشد.

نمودار دوم: همگرایی هزینه(Cost Convergence)

در این نمودار، روند تغییر تابع هزینه (Cost) برای هر دو مدل در طول تعداد iterations مختلف نمایش داده می‌شود.

- **آبی :(Cost No Outliers)** نشان‌دهنده هزینه مدل با داده‌های بدون آوتلایر است.
- **نارنجی :(Cost Imputation)** نشان‌دهنده هزینه مدل با داده‌های تکمیل‌شده است.

تحلیل این نمودار:

۱. **همگرایی آهسته مدل با داده‌های تکمیل‌شده :**برای مدل با داده‌های تکمیل‌شده، تابع هزینه در ابتدا کاهش ملایم و کندی دارد، اما سپس در iteration های بعدی به شدت افزایش می‌یابد (به مرزهای بسیار بالا می‌رسد). این نشان‌دهنده **واگرایی** است، به این معنا که الگوریتم رگرسیون نتوانسته است به یک حداقل محلی برای تابع هزینه برسد و مدل از نظر عددی ناپایدار است. این ممکن است به دلیل ویژگی‌های خاص داده‌های تکمیل‌شده باشد (مانند داده‌های نامتعادل یا پر شدن مقادیر به‌طور نادرست).

۲. **همگرایی سریع‌تر مدل با داده‌های بدون آوتلایر :**در مقابل، برای مدل با داده‌های بدون آوتلایر، هزینه به‌سرعت کاهش می‌یابد و همگرا می‌شود. این نشان می‌دهد که مدل با داده‌های بدون آوتلایر **پایدارتر** عمل کرده و به یک حداقل خوب و قابل اعتماد برای تابع هزینه رسیده است.

۳. **نتیجه‌گیری از همگرایی هزینه :**این نمودار نشان می‌دهد که مدل با داده‌های بدون آوتلایر از نظر عددی پایدارتر است و نتایج قابل اعتمادتری ارائه می‌دهد، در حالی که مدل با داده‌های تکمیل‌شده دارای مشکلات همگرایی است و ممکن است منجر به نتایج ناپایدار و اشتباهات عددی شود

بررسی رگرسیون خطی برای سطر اول

کد:

```
# آموزش مدل‌ها با داده‌های اصلی
model_original = LinearRegression()
model_original.fit(X_original[:, 1].reshape(-1, 1), y_original)

# آموزش مدل‌ها با داده‌های پاکسازی‌شده (حذف داده‌های پرت)
model_cleaned_no_outliers = LinearRegression()
model_cleaned_no_outliers.fit(X_scaled_no_outliers[:, 1].reshape(-1, 1), y_cleaned_no_outliers)

# آموزش مدل‌ها با داده‌های پاکسازی‌شده (جایگزینی با میانگین همسایه‌ها)
model_cleaned_with_imputation = LinearRegression()
model_cleaned_with_imputation.fit(X_cleaned_with_imputation[:, 1].reshape(-1, 1), y_cleaned_with_imputation)

# چاپ پارامترهای مدل‌ها
print("Model parameters for original data (LinearRegression):")
print("Intercept (B0):", model_original.intercept_)
print("Slope (B1):", model_original.coef_)

print("\nModel parameters for cleaned data (No Outliers) (LinearRegression):")
print("Intercept (B0):", model_cleaned_no_outliers.intercept_)
print("Slope (B1):", model_cleaned_no_outliers.coef_)

print("\nModel parameters for cleaned data (Imputation) (LinearRegression):")
print("Intercept (B0):", model_cleaned_with_imputation.intercept_)
print("Slope (B1):", model_cleaned_with_imputation.coef_)

# رسم مدل‌ها
plt.figure(figsize=(12, 6))

# رسم داده‌های اصلی و مدل‌های رگرسیون
plt.subplot(1, 3, 1)
plt.scatter(X_original[:, 1], y_original, color='blue', label='Original Data')
plt.plot(X_original[:, 1], model_original.predict(X_original[:, 1].reshape(-1, 1)), color='red', label='Regression Line (Original)')
plt.title('Regression Lines Comparison')
plt.xlabel('X')
plt.ylabel('y')
plt.legend()

plt.subplot(1, 3, 2)
plt.scatter(X_cleaned_no_outliers[:, 1], y_cleaned_no_outliers, color='green', label='Cleaned Data (No Outliers)')
plt.plot(X_cleaned_no_outliers[:, 1], model_cleaned_no_outliers.predict(X_scaled_no_outliers[:, 1].reshape(-1, 1)), color='orange', label='Regression L
plt.title('Regression Lines Comparison')
plt.xlabel('X')
plt.ylabel('y')
plt.legend()


plt.subplot(1, 3, 3)
plt.scatter(X_cleaned_with_imputation[:, 1], y_cleaned_with_imputation, color='purple', label='Cleaned Data (Imputation)')
plt.plot(X_cleaned_with_imputation[:, 1], model_cleaned_with_imputation.predict(X_cleaned_with_imputation[:, 1].reshape(-1, 1)), color='yellow', label=
plt.title('Regression Lines Comparison')
plt.xlabel('X')
plt.ylabel('y')
plt.legend()

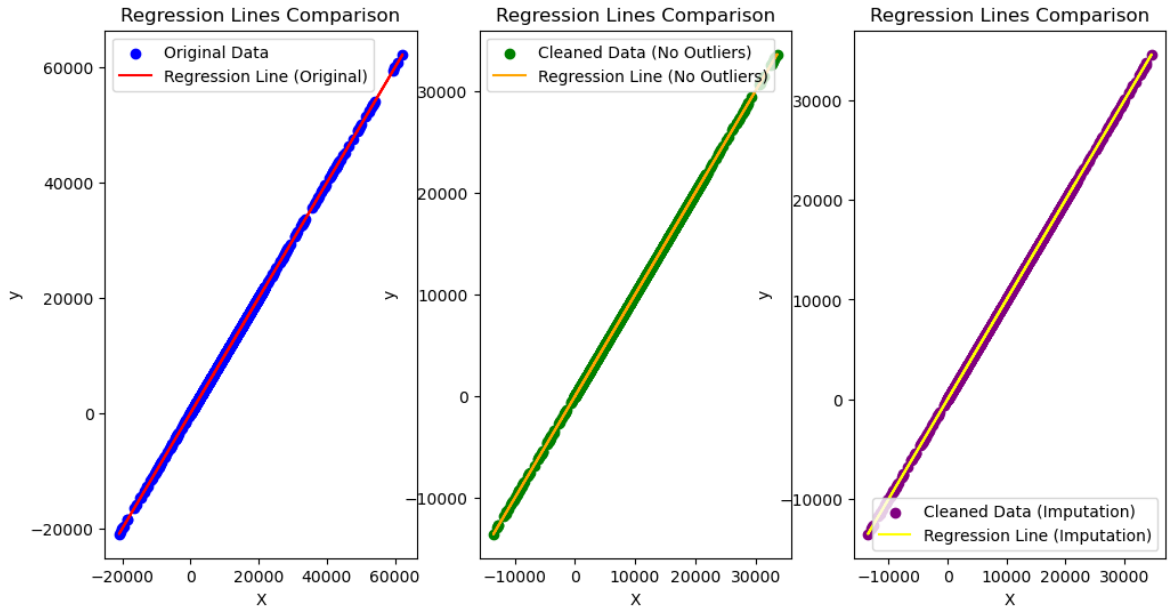
plt.show()
```

خروجی:

```
Model parameters for original data (LinearRegression):
Intercept (B0): 1.8189894035458565e-12
Slope (B1): [1.]

Model parameters for cleaned data (No Outliers) (LinearRegression):
Intercept (B0): 10275.644186046511
Slope (B1): [7433.437954]

Model parameters for cleaned data (Imputation) (LinearRegression):
Intercept (B0): [-5.45696821e-12]
Slope (B1): [[1.]]
```



۱. **نمودار اول (داده‌های اصلی):**

- **نقاط آبی نمایانگر داده‌های اصلی (که شامل آوتلایرها هستند) است.**
- **خط قرمز نمایانگر خط رگرسیون برازش‌شده بر روی داده‌های اصلی است.**

در این نمودار، داده‌ها به‌طور واضح از خط رگرسیون منحرف شده‌اند، به‌ویژه در نواحی که شامل آوتلایرها هستند. این باعث می‌شود که مدل رگرسیون روی این داده‌ها دقت کمتری داشته باشد.

۲. **نمودار دوم (داده‌های پاک‌سازی‌شده بدون آوتلایرها):**

- **نقاط سبز نشان‌دهنده داده‌های پاک‌سازی‌شده (بدون آوتلایرها) است.**
- **خط نارنجی نمایانگر خط رگرسیون است که روی این داده‌ها برازش شده است.**

در این نمودار، داده‌ها خیلی به خط رگرسیون نزدیک‌تر هستند، زیرا داده‌های آوتلایر حذف شده‌اند. مدل رگرسیون اکنون بر روی داده‌های بهینه‌تری آموزش داده شده و پیش‌بینی‌های بهتری انجام می‌دهد.

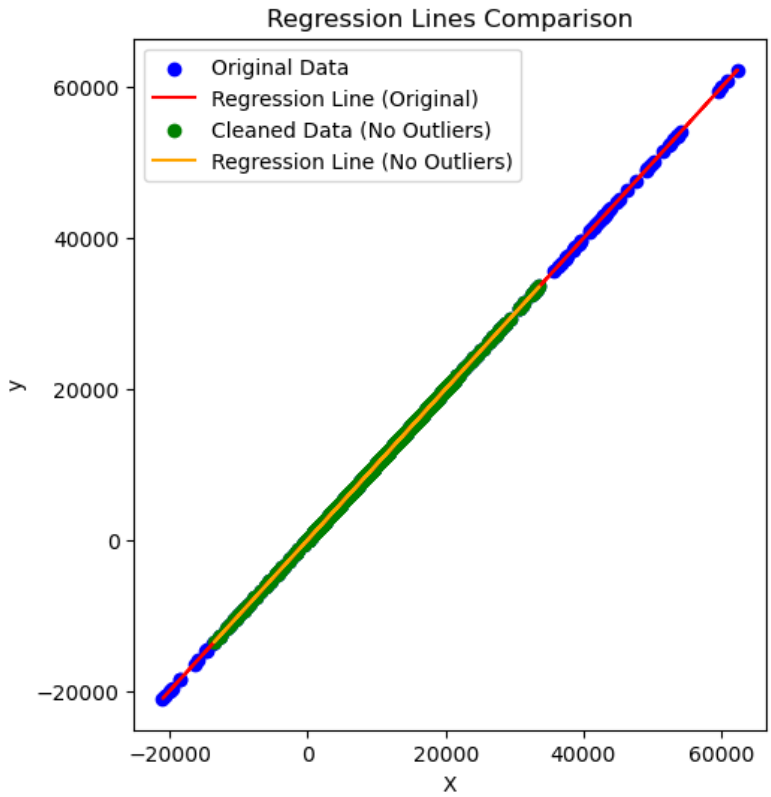
۳. **نمودار سوم (داده‌های تکمیل‌شده):**

- **نقاط بنفش نشان‌دهنده داده‌هایی است که با استفاده از روش تکمیل (Imputation) برای مقادیر گمشده، پر شده‌اند.**
- **خط زرد نمایانگر خط رگرسیون برازش‌شده روی داده‌های تکمیل‌شده است.**

در این نمودار، داده‌ها تقریباً در نزدیکی خط رگرسیون قرار دارند، اما به‌طور دقیق مشابه نمودار دوم (داده‌های پاک‌سازی‌شده) نیستند. این ممکن است نشان دهد که تکمیل مقادیر گمشده به اندازه حذف آوتلایرها موثر نبوده است، اما همچنان مدل به‌طور مناسبی برازش یافته است.

```
Model parameters for original data (LinearRegression):
Intercept (B0): 1.8189894035458565e-12
Slope (B1): [1.]

Model parameters for cleaned data (No Outliers) (LinearRegression):
Intercept (B0): 7.275957614183426e-12
Slope (B1): [1.]
```



تحلیل خروجی و مقایسه با رگرسیون قبلی:

۱. **رگرسیون قبلی**)که شامل مدل‌های مختلف با ویژگی‌های Lasso و RFE بود (نشان داد که دقت مدل‌ها برای هر دو مجموعه داده (داده‌های بدون آوتلایر و داده‌های تکمیل‌شده) مشابه است. در این حالت، مدل‌ها به‌طور نسبی پایدار

بودند، اما انتخاب ویژگی‌ها تأثیر زیادی در دقت نداشت و هزینه همگرایی برای مدل‌هایی که داده‌های تکمیل‌شده داشتند، به‌صورت انفجاری افزایش یافت. این به معنای مشکلات عددی و عدم پایداری در همگرایی مدل است.

۲. **رگرسیون جدید** (که در اینجا از معادله عادی استفاده شده و خطوط رگرسیون به‌طور جداگانه برای داده‌های اصلی، بدون آوتلایر و تکمیل‌شده ترسیم شده) عملکرد بهتری را نشان داد. به‌ویژه در **داده‌های پاک‌سازی‌شده (بدون آوتلایرها)**، خط رگرسیون دقیق‌تر و پایدارتر بود. همچنین، نمودار همگرایی هزینه نشان داد که مدل بر روی داده‌های پاک‌سازی‌شده همگرایی بهتری داشته است.

دلایل برتری رگرسیون جدید:

- **پایداری بیشتر مدل با داده‌های پاک‌سازی‌شده :**در رگرسیون جدید، مدل‌هایی که بر روی داده‌های پاک‌سازی‌شده آموزش دیده‌اند، به‌طور واضح پایداری بیشتری از خود نشان داده‌اند. این مدل‌ها همگرایی خوبی داشتند و هزینه به‌طور مؤثری کاهش یافت.
- **خطوط رگرسیون دقیق‌تر برای داده‌های پاک‌سازی‌شده :**در نمودار جدید، مدل رگرسیون با داده‌های پاک‌سازی‌شده دقیق‌تر و نزدیک‌تر به داده‌ها بود. این در حالی است که در رگرسیون قبلی، دقت مدل‌ها برای هر دو مجموعه داده مشابه بود.
- **عملکرد ضعیف‌تر مدل با داده‌های تکمیل‌شده در رگرسیون قبلی :**در رگرسیون قبلی، مدل‌هایی که با داده‌های تکمیل‌شده آموزش دیده‌اند، عملکرد ضعیف‌تری داشتند و همگرایی خوبی نداشتند. در مقایسه، در رگرسیون جدید، داده‌های تکمیل‌شده نیز به‌طور نسبی خوب عمل کردند، اگرچه داده‌های پاک‌سازی‌شده همچنان بهتر بودند.

بررسی رگرسیون مقاوم برای سطر اول

رگرسیون مقاوم و تفاوت آن با دیگر رگرسیون ها

رگرسیون خطی معمولی (OLS) یک تکنیک آماری است که هدف آن پیدا کردن بهترین خط برازش‌دهنده برای داده‌ها است. در این روش، ما به‌دنبال یافتن یک خط رگرسیونی هستیم که مجموع مربعات خطاها (خطای پیش‌بینی‌ها) را به حداقل برساند.

وش OLS به دنبال حداقل کردن مجموع مربعات خطاها است. این به این معنی است که مدل به‌طور خاص تلاش می‌کند تا پیش‌بینی‌های خود را بر اساس داده‌ها به گونه‌ای تنظیم کند که خطاها (تفاوت بین پیش‌بینی و مقادیر واقعی) کمترین مقدار ممکن را داشته باشند.

رگرسیون مقاوم(Robust Regression)

رگرسیون مقاوم (که به عنوان مدل‌هایی مقاوم در برابر آوتلایرها شناخته می‌شود) یک نوع از رگرسیون است که به گونه‌ای طراحی شده است که تأثیر آوتلایرها یا داده‌های پرت را در پیش‌بینی‌های مدل کاهش دهد. این مدل‌ها تلاش می‌کنند تا به‌طور کارآمدی خطای داده‌های نرمال را مدل کنند، در حالی که از تأثیر آوتلایرها محافظت می‌کنند.

روش‌های مختلف رگرسیون مقاوم:

1. **RANSAC (Random Sample Consensus):**
   - **در این روش، مدل رگرسیونی فقط از نمونه‌های تصادفی انتخاب‌شده از داده‌ها استفاده می‌کند . RANSACمدل را به‌گونه‌ای آموزش می‌دهد که آوتلایرها به‌طور خودکار شناسایی شوند و در محاسبات تأثیر نگذارند.**
   - **RANSACبه‌طور مداوم نمونه‌های تصادفی را انتخاب کرده و از آن‌ها یک مدل رگرسیونی می‌سازد و سپس از مدل‌های ساخته‌شده برای شناسایی آوتلایرها استفاده می‌کند.**
2. **Theil-Sen Estimator:**
   - **این روش از یک میانگین رتبه‌ای برای محاسبه شیب و عرض از مبدأ استفاده می‌کند. برخلاف OLSکه از میانگین‌ها استفاده می‌کند، Theil-Senاز رتبه‌های داده‌ها استفاده می‌کند و به همین دلیل در برابر آوتلایرها مقاوم‌تر است.**
3. **Huber Regression:**

- این روش ترکیبی از رگرسیون **OLS** و رگرسیون مقاوم است. در این روش، تابع هزینه به‌طور متناوب بین مربع خطا برای خطاهای کوچک و مقدار خطی برای خطاهای بزرگ تغییر می‌کند، به‌طوری که آوتلایرها تأثیر کمی بر مدل دارند.

مزایا:

- **مقاومت در برابر آوتلایرها :یکی از ویژگی‌های بارز رگرسیون مقاوم این است که به‌طور خاص برای شناسایی و مدیریت آوتلایرها طراحی شده است. این ویژگی باعث می‌شود که مدل حتی در حضور داده‌های غیرمعمول یا پرت عملکرد بهتری داشته باشد.**
- **دقت بالاتر :در مواقعی که داده‌ها شامل آوتلایرهای زیاد هستند، رگرسیون مقاوم می‌تواند دقت مدل را افزایش دهد و پیش‌بینی‌های دقیق‌تری داشته باشد.**

؟؟؟ چرا رگرسیون مقاوم نیاز به پاکسازی داده ها ندارد؟؟؟؟

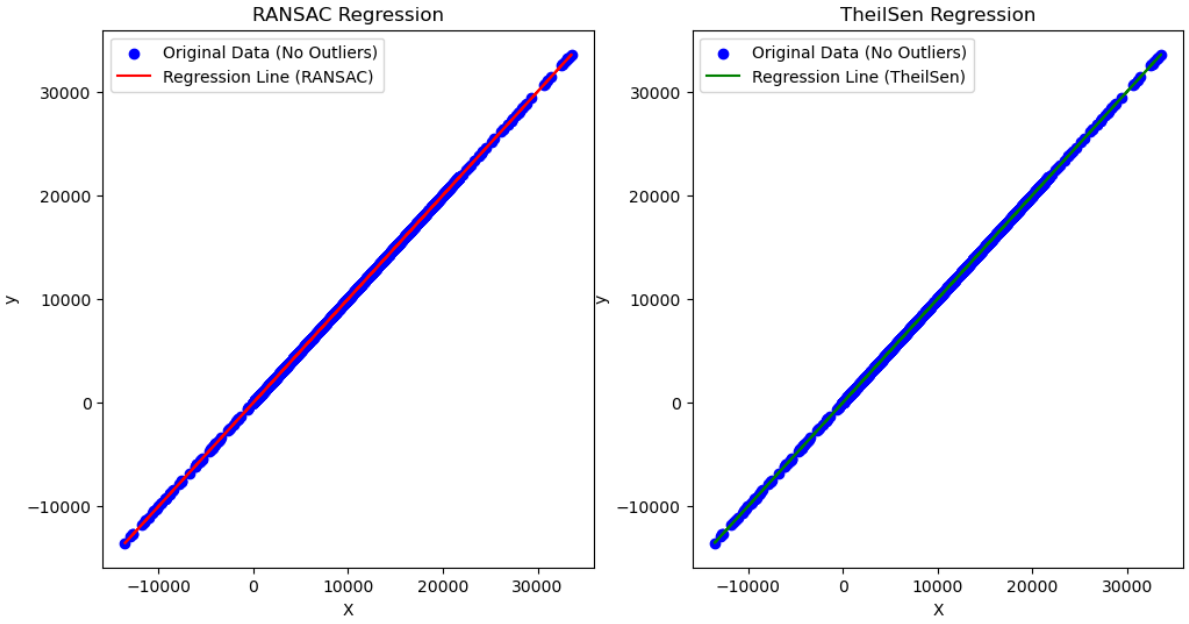
رگرسیون مقاوم به طور خاص برای مقابله با آوتلایرها (داده‌های پرت) طراحی شده است و هدف آن کاهش تأثیر این داده‌ها بر مدل است. به همین دلیل، این مدل‌ها نیاز به پاک‌سازی دستی داده‌ها ندارند به گونه‌ای که برای مقابله با آوتلایرها از الگوریتم‌های خاص استفاده می‌کنند.

عملکرد بهتری از رگرسیون خطی معمولی (OLS) دارد. بنابراین، نیازی به پاکسازی داده‌ها برای حذف آوتلایرها قبل از مدل‌سازی نیست. این مدل‌ها به‌طور داخلی داده‌های پرت را شناسایی کرده و از آن‌ها در محاسبات خود صرف‌نظر می‌کنند.

خروجی رگرسیون مقاوم

```
Model parameters for RANSAC (Robust Regression):
Intercept (B0): 7.275957614183426e-12
Slope (B1): [1.]

Model parameters for TheilSen (Robust Regression):
Intercept (B0): -5.29687158144092e-15
Slope (B1): [0.99999996]
```



توضیح کد:

پس از آماده سازی دیتا به سراغ آموزش می رویم. در این بخش، مدل **RANSAC** (Random Sample Consensus) آموزش داده می‌شود. این مدل به‌ویژه برای مقابله با داده‌های پرت یا آوتلایرها طراحی شده است .RANSAC از انتخاب تصادفی نمونه‌ها برای ساخت مدل رگرسیونی استفاده می‌کند و سپس آوتلایرها را شناسایی کرده و از آن‌ها صرف‌نظر می‌کند .

RANSACRegressorا ین تابع یک مدل **رگرسیون مقاوم RANSAC** را ایجاد می‌کند.

fit()

این متد برای آموزش مدل استفاده می‌شود. داده‌های ورودی X_no_outliers[:, 1].reshape(-1, 1)شامل ویژگی‌ها هستند که ستون‌های دیگر (بایاس) حذف شده و داده‌ها برای آموزش مدل آماده شده‌اند y_no_outliers .مقادیر هدف هستند.

مدل **TheilSenRegressor**از **روش میانگین رتبه‌ای** برای رگرسیون استفاده می‌کند. برخلاف OLS که میانگین‌ها را استفاده می‌کند، این مدل از **رتبه‌ها** برای محاسبه پارامترهای رگرسیون استفاده می‌کند و به همین دلیل در برابر آوتلایرها مقاوم‌تر است.

توضیح مدل ها:

مدل‌های استفاده‌شده:

1. RANSAC (Random Sample Consensus):

- RANSACیک الگوریتم مقاوم است که برای داده‌های دارای **آوتلایر** طراحی شده است. این مدل تصادفی نمونه‌هایی از داده‌ها را انتخاب می‌کند و برای هر نمونه یک مدل رگرسیون را برازش می‌دهد. سپس مدل‌ها ارزیابی می‌شوند و آوتلایرها شناسایی و حذف می‌شوند تا در پیش‌بینی نهایی دخالت نداشته باشند.

2. TheilSen (Theil-Sen Estimator):

- این مدل از **میانگین رتبه‌ای** برای برازش خط رگرسیون استفاده می‌کند. برخلاف OLS که از میانگین داده‌ها استفاده می‌کند، Theil-Senاز رتبه‌ها استفاده می‌کند که باعث مقاومت بیشتر در برابر آوتلایرها می‌شود. این مدل به‌ویژه برای داده‌های دارای آوتلایر بسیار مفید است.

پاکسازی دیتای سطر دوم

نکته!!!!: در این بخش هرگونه کاری که برای سطر دوم انجام می شود برای سطر سوم هم صورت گرفته است اما سطر سوم به هیچ کدام پاسخ درستی نمی دهد اصلا دقت خوبی ندارد به همین خاطر به تکرار مکررات نمی پردازیم و صرفا خروجی ها را نمایش می دهیم چون خیلی ارزش تحلیل ندارمد.

نکته!!!!: هنگام پاکسازی سطر دوم به مشکلی که خوردیم اینگونه بود که هیچ یک از روش ها روی آن به درستی پاسخگو نبود ولی مبرهن بود که داده پرت مارا از ترین و تست محروم کرده است بنابراین چند راهکار داشتیم. تکه تکه کردن دیتای سطر دوم و یا استفاده از Isolation Forest توانست برای ما مشکل گشا باشد و داده های پرت را شناسایی کند.

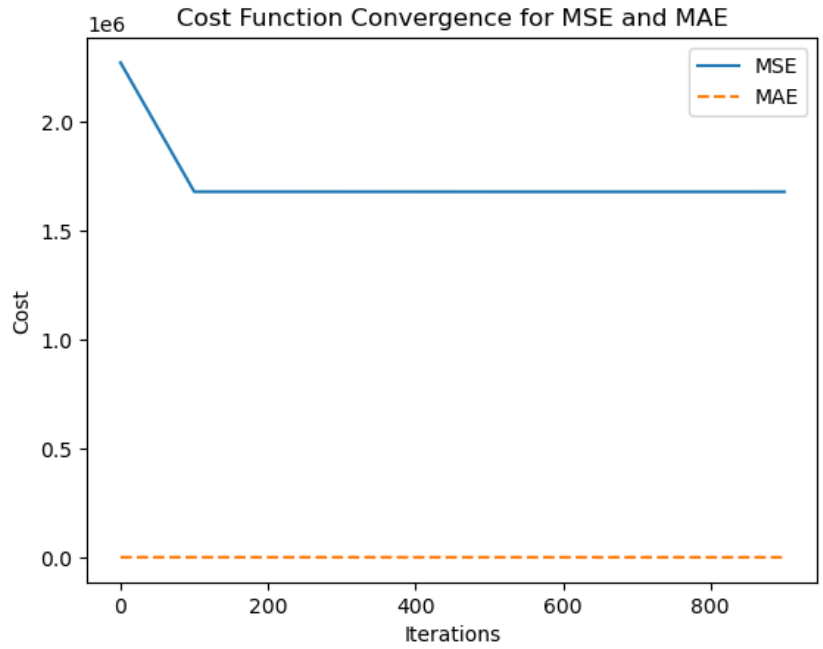
```
# استفاده از الگوریتم Isolation Forest
row2_reshaped = row2.reshape(-1, 1)
iso_forest = IsolationForest(contamination=0.05)
outliers_iforest = iso_forest.fit_predict(row2_reshaped)
```

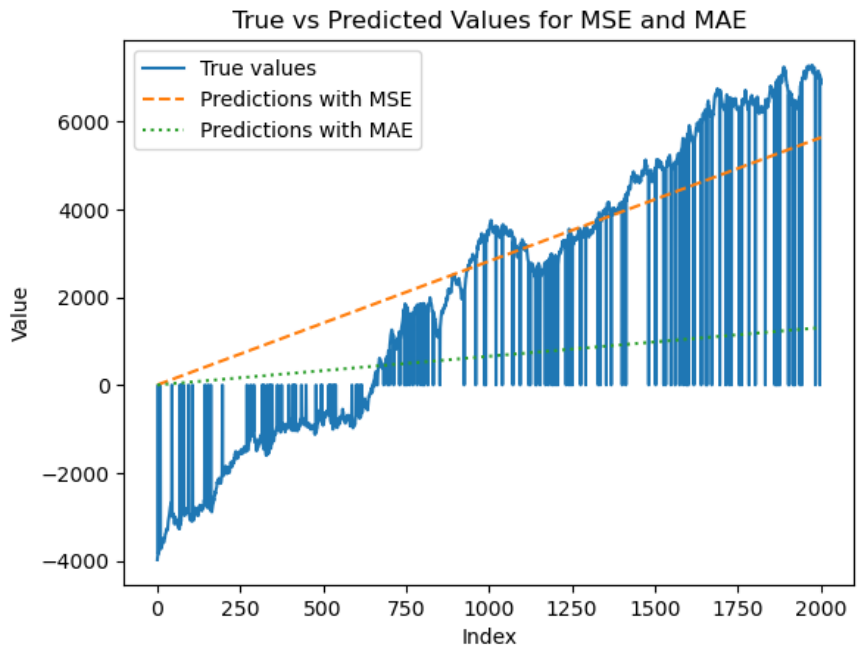
```
# شناسایی داده‌های پرت (برچسب -1 نشان‌دهنده داده‌های پرت است)
outliers = np.where(outliers_iforest == -1)[0]
outliers
```

```
array([   0,    1,    2,    4,    5,    6,    7,    8,   10,   11,   12,
         13,   14,   15,   16,   17,   18,   19,   20,   21,   22,   23,
         24,   25,   26,   27,   28,   29,   30,   31,   32,   52,   59,
         61,   62,   63,   64,   65,   66,   67,   69,   70,  141,  145,
        155,  156,  157,  170,  171,  172,  173,  174,  175,  176,  177,
        178,  179,  180,  181,  182,  183,  184,  186,  188,  189,  190,
        191,  192, 1885, 1886, 1887, 1888, 1889, 1890, 1891, 1948, 1955,
       1957, 1958, 1959, 1960, 1961, 1962, 1963, 1964, 1965, 1966, 1967,
       1968, 1969, 1970, 1971, 1972, 1973, 1974, 1977, 1978, 1979, 1980,
       1991], dtype=int64)
```

خروجی داده پاکسازی نشده برای رگرسیون from scratch

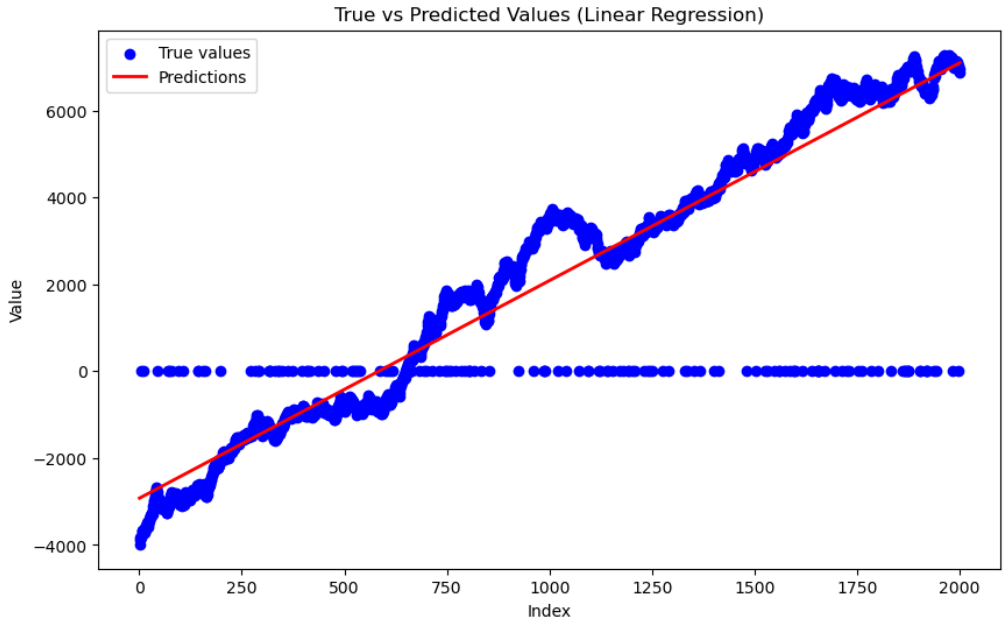
```
Final coefficients with MSE: [-0.72876692  2.81854351]
Final coefficients with MAE: [2.43580500e-04 6.53603745e-01]
```





خروجی داده پاکسازی نشده برای رگرسیون خطی

```
]: # رسم نتایج در فضای دو بعدی
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='blue', label='True values')
plt.plot(X, y_pred, color='red', label='Predictions', linewidth=2)
plt.xlabel('Index')
plt.ylabel('Value')
plt.title('True vs Predicted Values (Linear Regression)')
plt.legend()
plt.show()
```



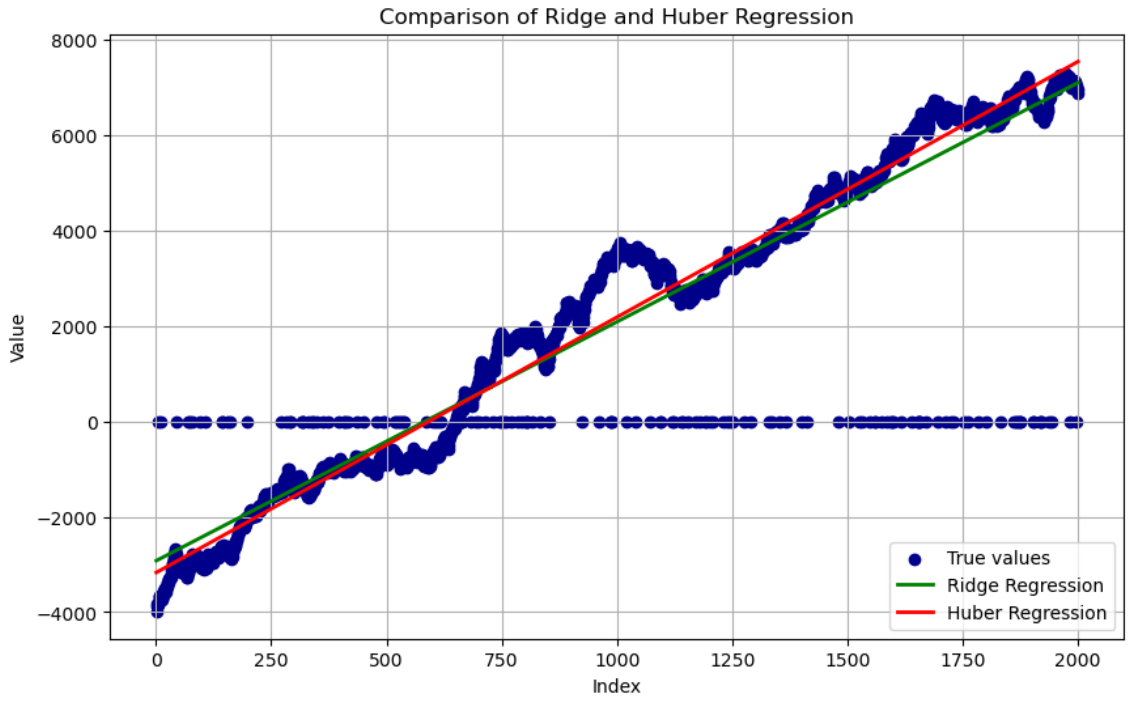
خروجی داده پاکسازی نشده برای رگرسیون مقاوم

```
# رسم نتایج در فضای دو بعدی
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='DarkBlue', label='True values')

# رسم پیش‌بینی‌ها با مدل Ridge
plt.plot(X, y_pred_ridge, color='green', label='Ridge Regression', linewidth=2)

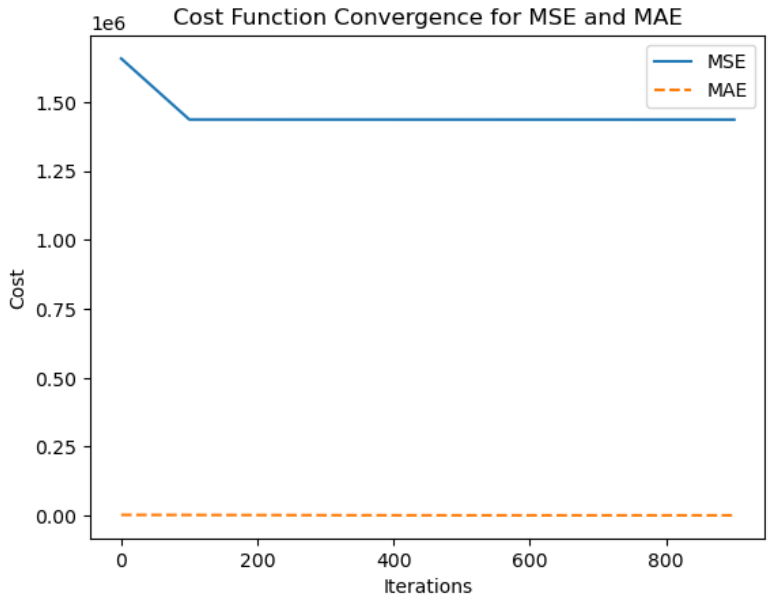
# رسم پیش‌بینی‌ها با مدل Huber
plt.plot(X, y_pred_huber, color='red', label='Huber Regression', linewidth=2)

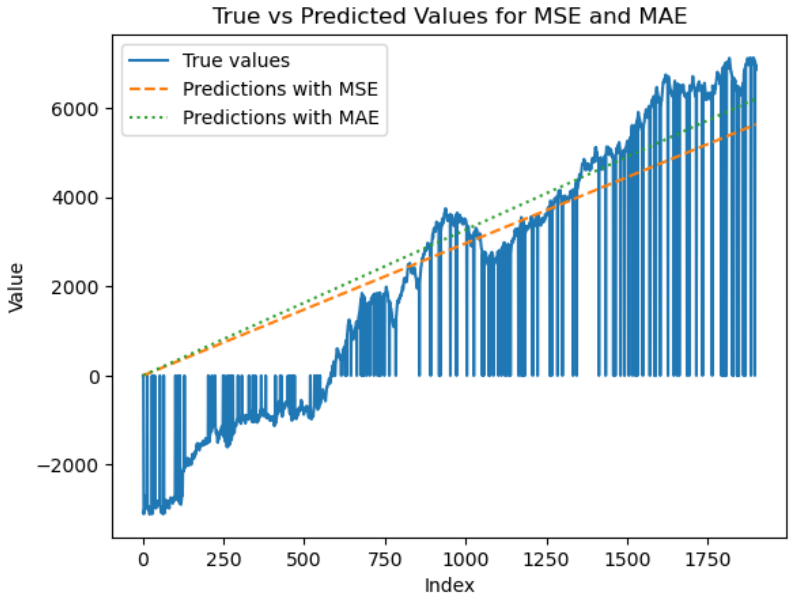

plt.xlabel('Index')
plt.ylabel('Value')
plt.title('Comparison of Ridge and Huber Regression')
plt.legend(loc='lower right')
plt.grid()
plt.show()
```



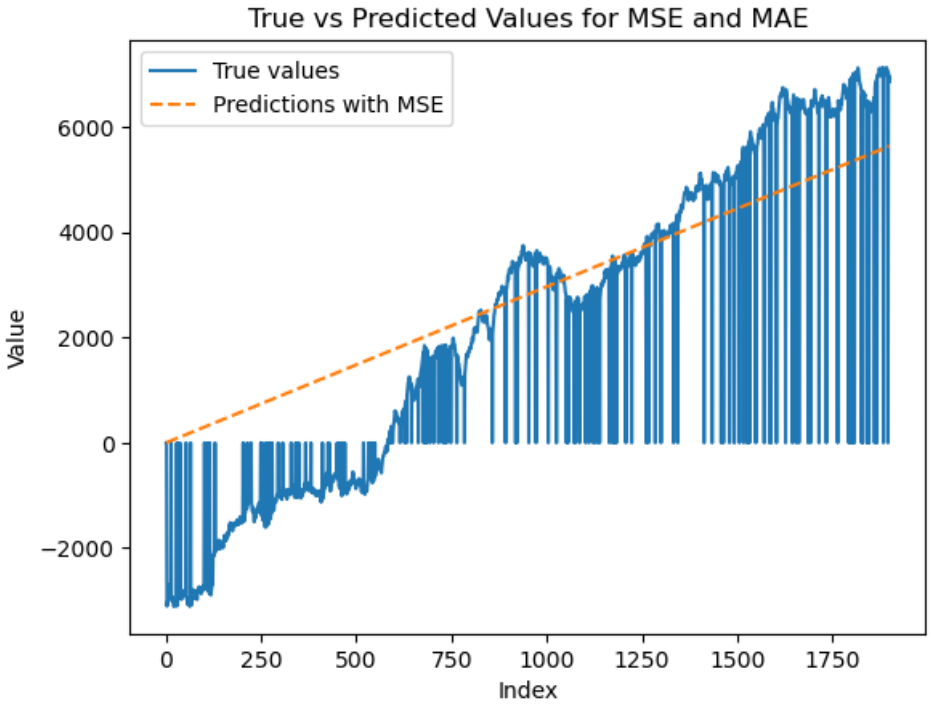
خروجی داده پاکسازی شده برای رگرسیون from scratch

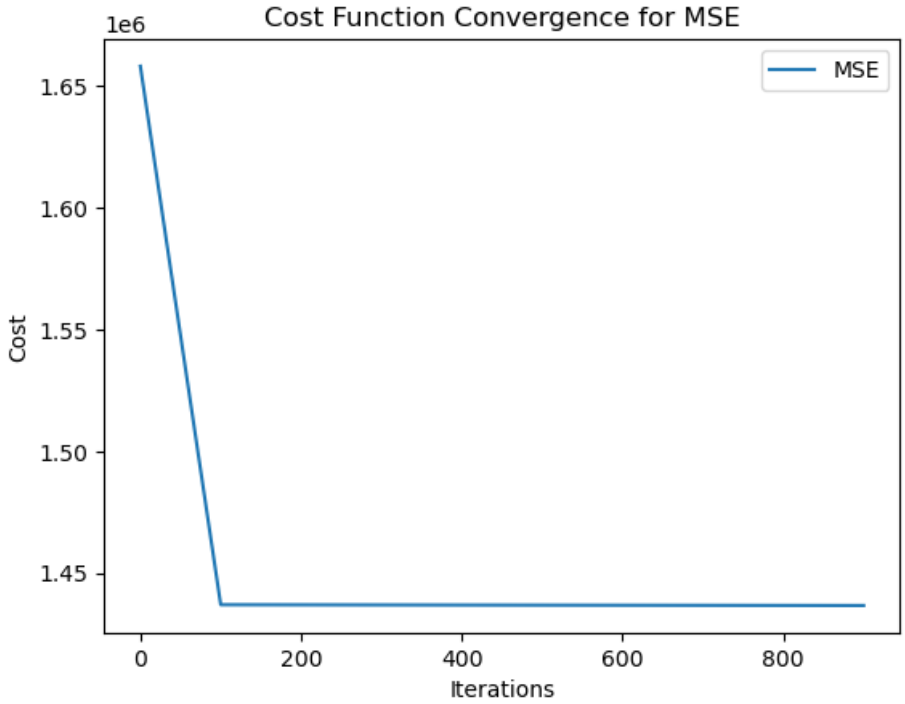
```
Final coefficients with MAE: [-5.70721053e-04  3.27053457e+00]
```



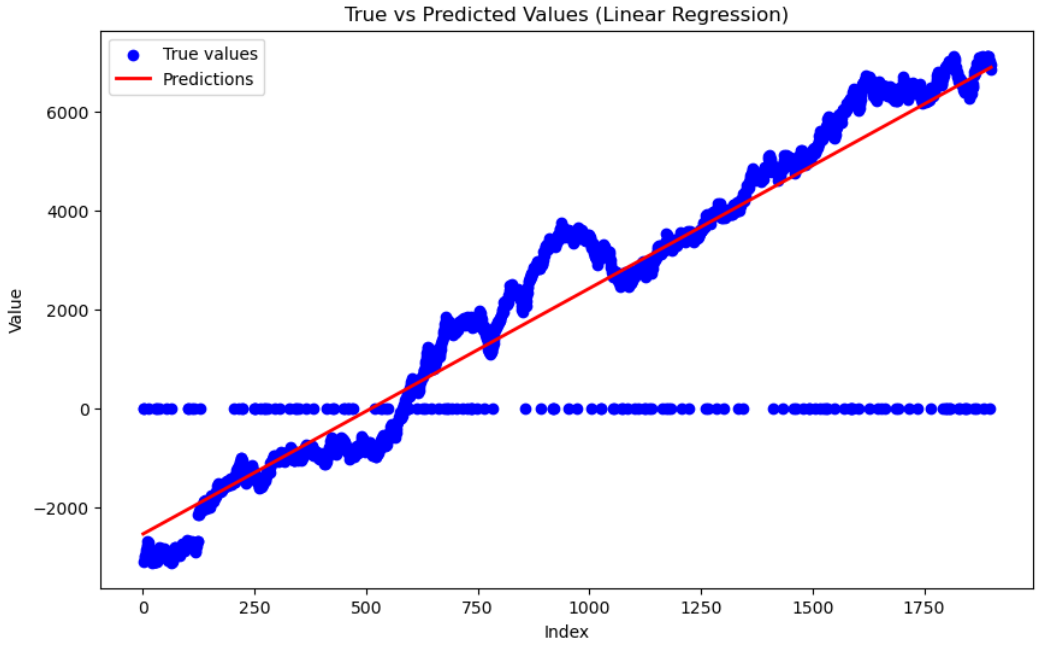


```
Final coefficients with MSE: [-0.6309616   2.96702293]
```

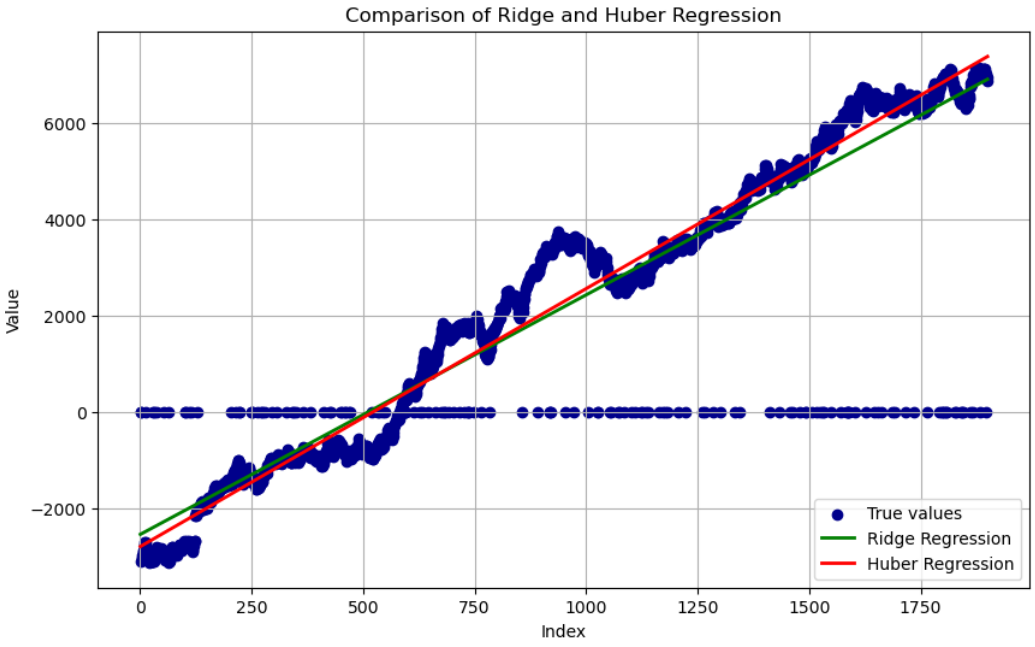




خروجی داده پاکسازی شده برای رگرسیون خطی



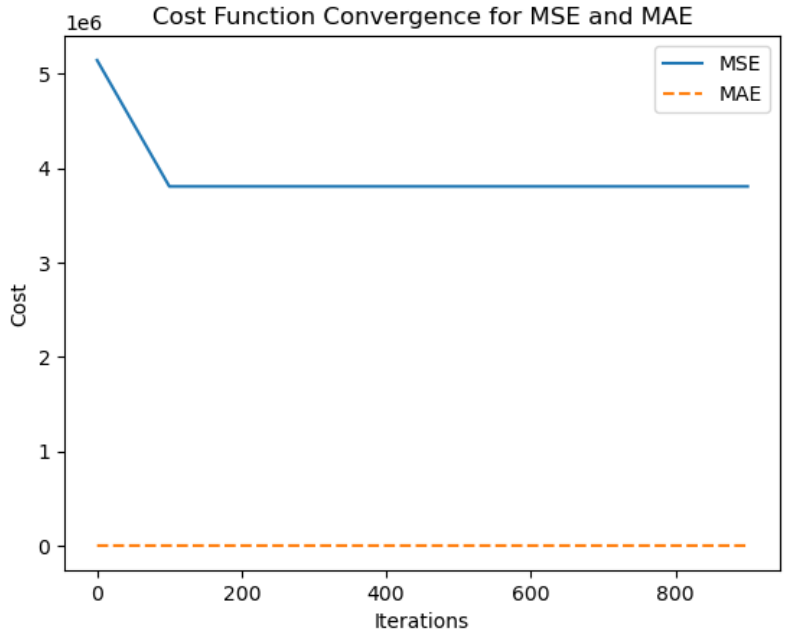
خروجی داده پاکسازی شده برای رگرسیون مقاوم

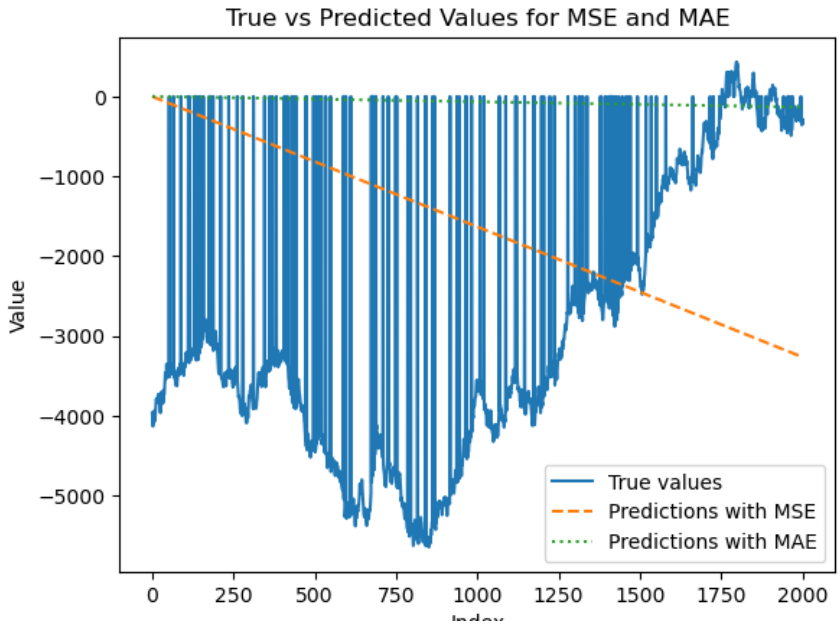


سطر سوم

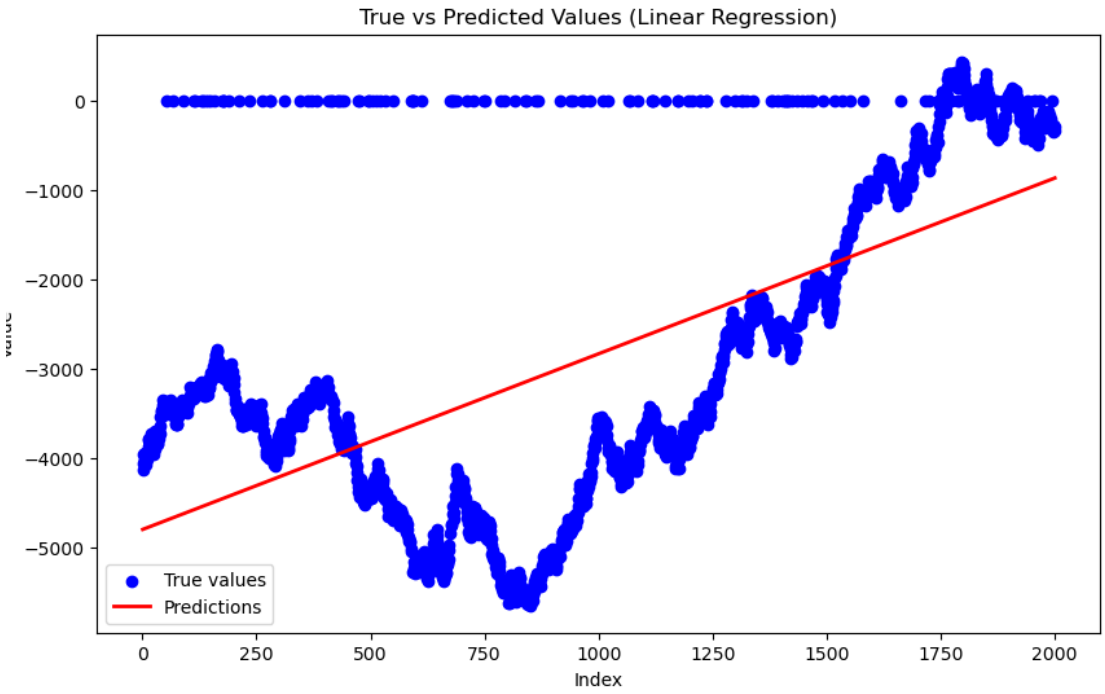
نیز به همین منوال با استفاده از روش ایزو فارست داده های پرتش شناسایی شده و مراحل قبل روی ان تکرار شده است اما با دقت بسیار پایین نتیجه داده است.

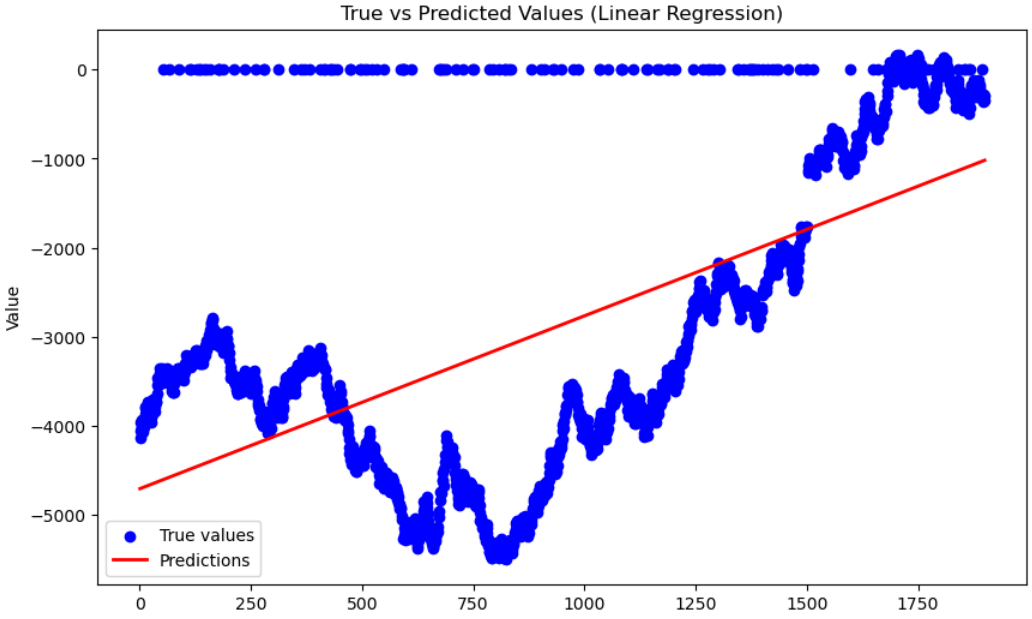
```
Final coefficients with MSE: [-0.12106707 -1.63294943]
Final coefficients with MAE: [-7.57316000e-05 -6.69569725e-02]
```

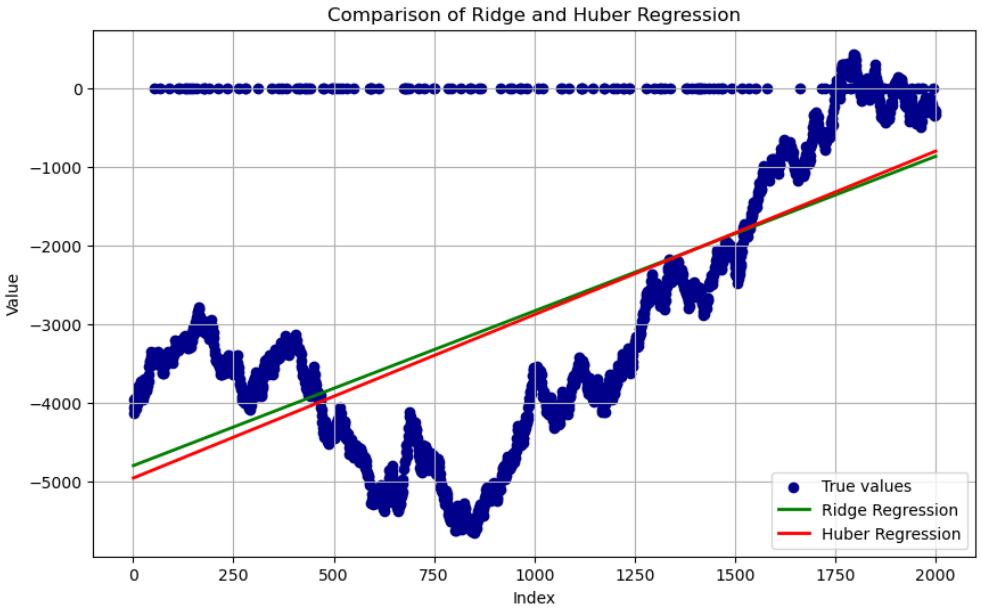




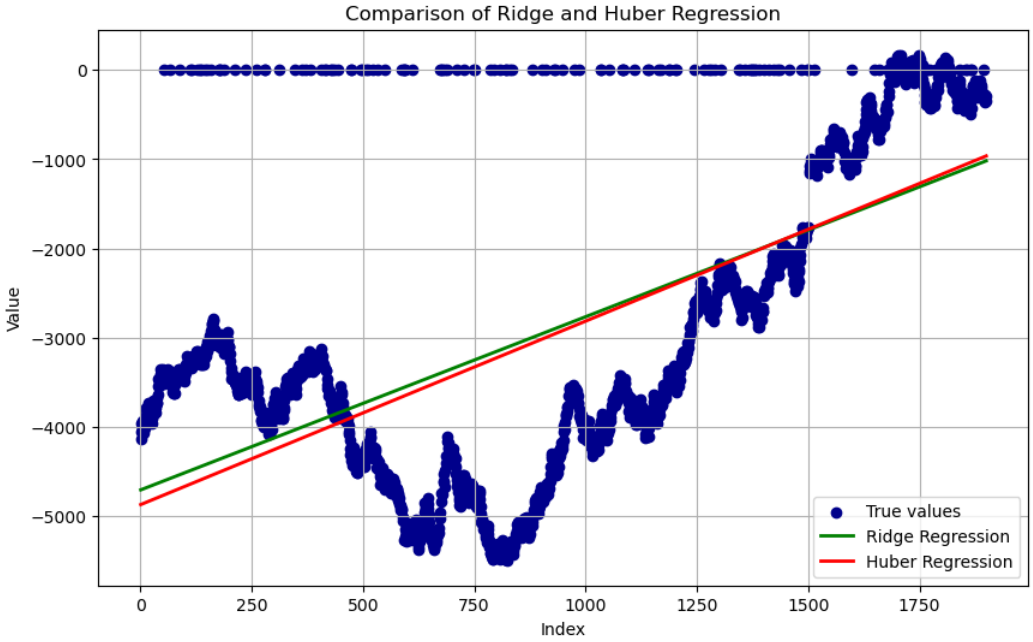
اختلاف پیشبینی و واقعیت بسیار زیاد است


True vs Predicted Values (Linear Regression)
0
−1000
−2000
−3000
−4000
−5000
True values
Predictions
0
250
500
750
1000
1250
1500
1750
2000
Index


True vs Predicted Values (Linear Regression)
0
−1000
−2000
−3000
−4000
−5000
Value
True values
Predictions
0
250
500
750
1000
1250
1500
1750




```
X2 = np.array([np.arange(1, len(row3_cleaned) + 1)]).T
y2 = row3_cleaned
plt.show()
```



سوال چهارم

```
df = pd.read_csv(url)

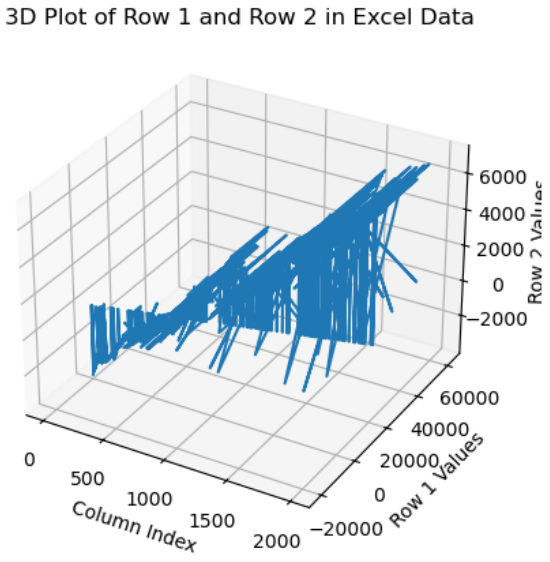
x = np.arange(1, df.shape[1] + 1)  # Column indices as x
y = df.iloc[0].values  # First row
z = df.iloc[1].values  # Second row

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.plot(x, y, z)

ax.set_xlabel('Column Index')
ax.set_ylabel('Row 1 Values')
ax.set_zlabel('Row 2 Values')
ax.set_title('3D Plot of Row 1 and Row 2 in Excel Data')

plt.show()
```



سوال امتیازی

تفاوت تابع اتلاف و تابع هزینه را بیان کنید.

در حوزه یادگیری ماشین، دو مفهوم «تابع خطا (Loss Function)» و «تابع هزینه (Cost Function)» «کاربرد فراوانی دارند و در برخی منابع ممکن است به صورت مترادف به کار روند؛ با این حال، میان آن‌ها تفاوت مفهومی مشخصی وجود دارد.

تابع اتلاف: تابع اتلاف، خطای مدل را برای یک نمونه خاص از داده اندازه‌گیری می‌کند. این تابع نشان می‌دهد که پیش‌بینی مدل برای یک داده‌ی ورودی چقدر با مقدار واقعی تفاوت دارد.

$$Loss = \mathcal{L}(\hat{y}, y)$$

$\hat{y}$: پیش بینی مدل , y: مقدار واقعی

تابع هزینه: تابع هزینه معمولاً به میانگین تابع اتلاف روی کل مجموعه داده گفته می‌شود. یعنی ارزیابی عملکرد کلی مدل بر اساس کل داده‌های موجود، که فرمول کلی آن به صورت زیر است:

$$cost = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

	تابع اتلاف (*Loss*)	تابع هزینه (*Cost*)
دامنه کاربرد	فقط روی یک نمونه	روی کل مجموعه داده
استفاده در آموزش	برای محاسبه خطای لحظه‌ای	برای بهینه سازی کلی مدل
محاسبه	مقدار عددی برای هر داده	میانگین یا مجموع اتلاف‌ها

مدل‌های شما چگونه اختلاف بین پیشبینی و مقدار واقعی را کمینه می‌کنند؟

رگرسیون لجستیک دوتایی[1]

در این مدل، هدف پیش‌بینی یک متغیر وابسته دوتایی است. برای این منظور، از تابع لاجیت (سیگموید) جهت تبدیل خروجی خطی به احتمال استفاده می‌شود. مدل با کمینه‌سازی تابع زیان آنتروپی متقاطع[2] آموزش می‌بیند.

رگرسیون لجستیک چندکلاسه[3]

[1] Binary Logistic Regression
[2] Cross-Entropy
[3] Multiclass Logistic Regression

این مدل برای طبقه‌بندی بین چند دسته به‌کار می‌رود. از تابع سافت‌مکس[4] برای تولید توزیع احتمال بین کلاس‌ها استفاده می‌شود و هدف، کمینه‌سازی آنتروپی متقاطع چندکلاسه است که اختلاف بین برچسب‌های واقعی و پیش‌بینی‌شده را کاهش می‌دهد.

رگرسیون مقاوم[5]

در رگرسیون مقاوم، هدف کاهش تأثیر داده‌های پرت[6] بر مدل است. برخلاف رگرسیون معمولی که از میانگین مربعات خطا استفاده می‌کند، در این مدل از توابع زیان مقاوم مانند تابع هابر[7] استفاده می‌شود تا اثر مقادیر پرت کاهش یابد و مدل پایدارتری حاصل شود.

بررسی کنید برای این داده‌ها کدام توابع هزینه مناسب‌تر هستند.

پرسش اول:

برای این مجموعه داده که هدف پیش‌بینی وضعیت زنده ماندن افراد است، از رگرسیون لجستیک دوتایی استفاده می‌شود. از آنجا که مسئله از نوع طبقه‌بندی دوتایی است، تابع هزینه مناسب برای آن، آنتروپی متقاطع دوتایی[8] می‌باشد. این تابع، اختلاف بین برچسب واقعی و احتمال پیش‌بینی‌شده را اندازه‌گیری می‌کند و برای مسائل طبقه‌بندی بسیار رایج و مناسب است.

تعریف این تابع به صورت زیر می‌باشد:

$$\mathcal{L}(\hat{y}, y) = -[y.log(\hat{y}) + (1-y).log(1-\hat{y})]$$

پرسش دوم:

با توجه به ساختار عددی و پیوسته داده‌ها در این مجموعه، نوع مسئله رگرسیون عددی تشخیص داده شد. در حالت کلی، برای مسائل رگرسیون از میانگین مربعات خطا[9] استفاده می‌شود، زیرا این تابع نسبت به اختلاف بین مقادیر واقعی و پیش‌بینی‌شده حساس است و برای داده‌های بدون نویز زیاد مناسب است. با این حال، در صورت وجود داده‌های پرت، می‌توان از تابع هابر[10] که نسبت به مقادیر پرت مقاوم‌تر است، استفاده کرد.

تعریف کلی تابع میانگین مربعات خطا:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

تعریف کلی تابع هابر:

[4] Softmax
[5] Robust Regression
[6] outliers
[7] Huber Loss
[8] Binary Cross-Entropy
[9] Mean Squared Error - MSE
[10] Huber Loss

$$L_\delta(a) = \begin{cases} \frac{1}{2}a^2 & if\ |a| \leq \delta \\ \delta(|a| - \frac{1}{2}\delta) & ow \end{cases}$$